

PYTHON PROGRAMMING

Contents

WHAT IS PYTHON?.....	6
TERMS AND DEFINITIONS :	6
CREATING CODE WITH PYTHON:.....	8
KEYWORDS:.....	9
ARITHMETIC OPERATORS:	9
ORDER OF OPERATIONS:	9
VARIABLE SCOPE:	10
LOCAL VARIABLE:	10
ENCLOSED VARIABLE:	10
GLOBAL:	10
BUILT-IN:	10
ARGUMENTS:.....	11
POSITIONAL ARGUMENTS:	11
DEFAULT ARGUMENTS	11
KEYWORD ARGUMENTS	11
ARBITRARY ARGUMENTS (*args & **kwargs).....	11
COMMAND-LINE ARGUMENTS:	12
FORMAT SPECIFIERS:.....	12
DATATYPES:	13
CHECK TYPE:	13
TYPECASTING:	13
STRINGS:.....	13
SINGLE AND DOUBLE QUOTES:.....	13
STRING METHODS:	14
CHANGING STRINGS:.....	15
STRING INDEXING CHEAT SHEET:	15
FINDING INDEX:	15
FINDING SUBSTRINGS:	16
HOW TO SLICE STRINGS:	16
HOW TO JOIN STRINGS :.....	16

INT:	16
FLOAT:	17
NAMING RULES AND CONVENTIONS:	17
VARIABLES ANNOTATED BY TYPE:	18
UNICODE:	18
FUNCTIONS:	19
USER-DEFINED FUNCTIONS:	19
BUILT-IN FUNCTIONS:	20
CONDITIONALS:	20
COMPARISON OPERATORS WITH EQUATIONS:	20
LOGICAL OPERATORS:	21
IF/ELSE/ELIF:	21
TERNARY OPERATOR:	22
MATCH STATEMENT:	22
LOOPS:	22
WHILE LOOP:	22
FOR LOOP:	23
RECURSION:	25
COLLECTION:	25
LISTS []:	26
LISTS METHODS:	26
MODIFYING LISTS:	27
ITERATING OVER LISTS AND TUPLES:	27
LIST COMPREHENSIONS:	28
2D LISTS:	30
SETS {}:	30
SETS METHODS:	30
TUPLES ():	30
TUPLES METHODS:	30
DICT ({key: value}):	31
DICT METHODS:	31
OOP:	34
INSTANCE VS CLASS VARIABLES:	34
SINGLE-INHERITANCE:	35
MULTIPLE & MULTI-LEVEL INHERITANCE:	36

POLYMRPHISM:	36
METHODS :	38
INSTANCE METHODS:	38
CLASS METHODS:	38
STATIC METHODS:	40
DUNDER METHODS:	40
CONSTRUCTOR:	41
DECORATORS:	42
PROPERTY DECORATOR:	42
FILE HANDLE I/O (.txt .csv.json):	44
FILE & DIRECTORY METHODS (OS AND OS.PATH):	44
FILE DETECTION:	45
FILE WRITING/APPENDING:	46
CSV (Comma Separated Values):	48
✦ CSV FILE WRITING:	48
✦ READING A CSV FILE (.csv)	49
LAMDA:	51
BINARY FILE:	51
EXCEPTIONS HANDLING:	52
SYNTAX ERRORS:	52
RUNTIME ERRORS:	52
TRY AND EXCEPT:	52
EXAMPLES:	52
DEBUGGING AND TROUBLESHOOTING:	54
WHAT IS TROUBLESHOOTING?	54
WHAT IS DEBUGGING?	54
PROBLEM SOLVING STEPS:	55
LIBRARIES:	55
HOW DO YOU LOAD A MODULE INTO YOUR OWN PROGRAM?	55
RANDOM MODULE:	55
STATISTICS MODULE:	56
SYS MODULE:	56
APIs (REQUESTS MODULE):	56
DATE-TIME MODULE:	58

MULTI-THREADING MODULE:	58
HOW TO INSTALL LIBRARY?	59
HOW TO CREATE YOUR OWN LIBRARY:	59
UNIT TEST:	60
ASSERT COMMAND:	60
PYTEST:	61
ORGANIZING TEST FOR MORE DETAILED TEST:	61
RAISE COMMAND:	62
TESTING STRINGS:	62
ORGANIZING TESTS INTO FOLDERS:	63
REGULAR EXPRESSION:	63
RE.SEARCH LIBRARY:	63
COMMON REGULAR EXPRESSION SYMBOLS & PATTERNS:	63
EMAIL VALIDATION:	64
ESCAPE CHARACTER and RAW STRINGS:	64
START AND END ANCHORS ^ \$:	65
COMPLIMENTING:	65
NOT COMPLIMENTING:	66
OR EXPRESSION:	67
FLAGS:	67
? EXPRESSION:	68
EMAIL VALIDATION EXPRESSION:	68
CLEANING UP USER INPUT:	68
WALRUS:	70
EXTRACTING USER IN PUT	70
REMOVE PREFIX:	70
SUB LIBRARY:	71
?: EXPRESSION:	72
FINAL PROJECT:	72
PRACTICE QUESTIONS:	73
TIP CALCULATOR:	73
TIME MODULE:	73
INTEREST CALCULATOR:	74
SHOPPING CART PROGRAM:	74
NUMPAD:	75

QUIZ GAME:	75
GUESS GAME:	76
SLOT MACHINE:	76
ENCRYPTION:	77
HANGMAN GAME:	78
ALARM CLOCK:	79
ROCK/PAPER/SCISSOR:	80
DICE ROLL:	81
BANK MANAGER:	82

WHAT IS PYTHON?

Python is widely used in the IT field, including IT support, system administration, web development, machine learning, data analytics, and more. Python can be used to calculate statistics, run your e-commerce site, process images, interact with web services, and do a whole host of other tasks. Python instructions resemble the English language, which is what makes it easier to learn and understand when compared to other programming languages.

Python is:

- a general purpose scripting language;
- a popular language used to code a variety of applications;
- a frequently used tool for automation;
- a cross-platform compatible language;
- a beginner-friendly language.

Python is not:

- a platform-specific / OS-specific scripting language;
- a client-side scripting language;
- a purely object-oriented programming language.

TERMS AND DEFINITIONS :

Automation: The process of replacing a manual step with one that happens automatically

Client-side scripting language: Primarily for web programming; the scripts are transferred from a web server to the end-user's internet browser, then executed in the browser

Code editors: Tools to provide features, including syntax highlighting, automatic indentation, error checking, and autocompletion

Computer program: A step-by-step list of instructions that a computer follows to reach an intended goal

Functions: A reusable block of code that performs a specific task

IDE: A software application that provides comprehensive facilities for software development

Interpreter: The program that reads and executes code

Input: Information that is provided to a program by the end user

Logic errors: Errors in code that prevent it from running correctly

Machine language: Lowest-level computer language. It communicates directly with computing machines in binary code (ones and zeros)

Object-oriented programming language: Most coding elements are considered to be objects with configurable properties

Output: the end result of a task performed by a function or computer program

Platform-specific scripting language: Language used by system administrators on those specific platforms

Programming: The process of writing a program to behave in different ways

Programming code: A set of written computer instructions, guided by rules, using a computer programming language

Programming languages: Language with syntax and semantics to write computer programs

Python: A general purpose programming language

Python interpreter: Program that reads and executes Python code by translating Python code into computer instructions

Script: Often used to automate specific tasks

Semantics: The intended meaning or effect of statements, or collections of words, in both human and computer languages

Syntax: The rules for how each statements are constructed in both human and computer languages

Variables: These are used to temporarily store changeable values in programming code

Bugs: are a natural part of coding.

Comments: are a way for programmers to track what they are doing in their programs and even inform others about their intentions for a block of code. In short, they are notes for yourself and others who will see your code!

Pseudocode: is an important type of comment that becomes a special type of to-do list, especially when you don't understand how to accomplish a coding task.

Argument and Parameters: An argument is the value passed to a function when it's called. Fundamentally, parameters are the variables inside a function's parentheses. Arguments provide values for those parameters

CREATING CODE WITH PYTHON:

You will be presented with >>> in the terminal window. You can then run live, interactive code. You could type 1+1, and it will run that calculation. This mode will not commonly be used during this course.

\$ Code hello.py (creates file)

\$ Python hello.py(runs file)

\$ clear (clear screen)

\$ls (lists all of files in folder)

\$cp hello.py goodbye.py(copy file)

\$mv goodbye.py farewell.py (renames file)

\$rm (deletes file)

\$mkdir folder (creates folder)

\$cd folder/ access folders

folder/\$ code farewell.py (create file in folder)

folder/\$ mv farewell.py..(default folder move)

\$rmdir folder (deletes folder)

KEYWORDS:

A keyword is a reserved word in a programming language that performs a specific purpose.

Values: **True, False, None**

Conditions: **if, elif, else**

Logical operators: **and, or, not**

Loops: **for, in, while, break, continue**

Functions: **def, return**

ARITHMETIC OPERATORS:

x + y Addition + operator returns the sum of x plus y

x - y Subtraction - operator returns the difference of x minus y

x * y Multiplication * operator returns the product of x times y

x / y Division / operator returns the quotient of x divided by y

xy** Exponent ** operator returns the result of raising x to the power of y

x2** Square expression returns x squared (**return pow(n,2)**)

x3** Cube expression returns x cubed

x(1/2)** Square root ($\frac{1}{2}$) or (0.5) fractional exponent operator returns the square root of x

x // y Floor division operator returns the integer part of the integer division of x by y

x % y Modulo operator returns the remainder part of the integer division of x by y

ORDER OF OPERATIONS:

The order of operations are to be calculated from left to right in the following order:

1. **Parentheses** (), { }, []
2. **Exponents** x^y (**x**y**)

3. **M**ultiplication * and **D**ivision /
4. **A**ddition + and **S**ubtraction -

You might find the **PEMDAS** mnemonic device to be helpful in remembering the order.

VARIABLE SCOPE:

Definition: where a variable is visible and accessible

Scope resolution order=(LEGB) Local->Enclosed->Global->Built-in

LOCAL VARIABLE:

it is **variable** defines inside functions, cannot be used by other functions

```
def func1():
    a=1
    print(a)
```

ENCLOSED VARIABLE:

In Python, an enclosed variable refers to a variable that is defined in an enclosing (non-global) scope and is accessed inside a nested function. Variables in an outer function (but not global).

```
def func1():
    x=1# enclosed
    def func2():
        x=2 #local
        print(x)
    func2()
func1() #answer is 2
```

```
def func1():
    x=1# enclosed
    def func2():
        print(x) #no local found
    func2()
func1() #answer is 1
```

GLOBAL:

Outside the functions, Variables defined at the module level.

```
a=3
def func1():
    print(a)
def func2():
    print(a)
func1()
func2()
```

BUILT-IN:

```
#from math import e
#e is built-in variable
```

ARGUMENTS:

POSITIONAL ARGUMENTS:

Arguments are passed in the order they appear in the function definition.

```
def greet(name, age):
    print(f'Hello, {name}! You are {age} years old.')

greet("Alice", 25) # ☒ Correct usage: arguments match parameter order

# greet(25, "Alice") # ☒ Incorrect order (can cause confusion)
```

DEFAULT ARGUMENTS

Default values reduce the number of required arguments.

```
def net_price(list_price, discount=0, tax=0.05): # Default values
    return list_price * (1 - discount) * (1 + tax)

print(net_price(500)) # ☒ Uses default discount and tax
print(net_price(500, 0.1, 0)) # ☒ Overrides default values
```

KEYWORD ARGUMENTS

Named arguments allow flexibility (order doesn't matter).

```
def hello(greeting, title, first_name):
    print(f'{greeting}, {title} {first_name}!')

hello("Hello", first_name="Jacob", title="Mr") # ☒ Order doesn't matter
```

☒ Positional arguments **must**** come before keyword arguments**

```
hello(first_name="Jacob", "Hello", title="Mr") # ☒ Will raise an error
```

ARBITRARY ARGUMENTS (*args & **kwargs)

***args :** Allows a function to accept any number of positional arguments. These arguments are packed into a tuple inside the function.

****kwargs :** Allows a function to accept any number of keyword arguments. These arguments are packed into a dictionary inside the function.

Example using *args

```
def add(*args):
    addition=0
    for arg in args:
        addition+=arg
    return addition
print(add(1,2,3,4))
```

Example using **kwargs

```
def address(**kwargs):
    for key, value in kwargs.items():
```

```
print(f"{key}: {value}")

address(street="123 Main St", city="New York", zip="10001")
# Output:
# street: 123 Main St
# city: New York
# zip: 10001
```

COMMAND-LINE ARGUMENTS:

Command-line arguments allow users to provide inputs to a Python script when executing it from the terminal or command prompt. These arguments are accessed using `sys.argv`, which is a list where:

- `sys.argv[0]` → The script name
- `sys.argv[1]` and onwards → The actual arguments provided

```
import sys
# Convert command-line arguments to integers and sum them
numbers = [int(arg) for arg in sys.argv[1:]] # Skip script name
print(f"Sum: {sum(numbers)}")
```

☒ END & SEP PARAMETERS IN PRINT

```
print("Hello", end=" ") # No newline
print("World!") # Prints: Hello World!

print("A", "B", "C", sep="-") # Prints: A-B-C
```

FORMAT SPECIFIERS:

format a value based on flags inserted {value:flags}

#:(number)f=round to many decimal places

```
price=3.14159
print(f"price value is:{price:.2f}")
```

#:(number)=allocate that many spaces

```
print(f"space allocation:{price:10}")
print(f"0 padded:{price:010}")
```

#:<=left justify

```
print(f"left justify:{price:<10}")
```

#:>=right justify

```
print(f"right justify:{price:>10}")
```

#:^=center align

```
print(f"center align:{price:^10}")
```

#:+= use a plus sign to indicate positive value

```
print(f"price value is:{price:+}")
```

#:= =place sign to left most position

#: =insert a space before positive numbers

```
print(f"price value is:{price: }")
```

#:, =comma separator

```
price2=3000.132333
print(f"price value is:{price2:,}")
```

```
print(f"price value is:{price2:+,.2f}")
```

DATATYPES:

Text Type:	Str
Numeric Types:	int, float, complex
Sequence Types:	list, tuple, range
Mapping Type:	Dict
Set Types:	set, frozenset
Boolean Type:	Bool
Binary Types:	bytes, bytearray, memoryview
None Type:	NoneType

CHECK TYPE:

```
print(type("a"))
```

TYPECASTING:

IMPLICIT TYPE CONVERSION: Also known as ‘automatic type conversion’. Done by the compiler on its own, without any external trigger from the user.

EXPLICIT TYPE CONVERSION: This process is also called type casting and it is user-defined. Here the user can typecast the result to make it of a particular data type.

EXAMPLES:

```
print("The area of the triangle is: " + str(area))
x = int(1) # x will be 1
y = int(2.8) # y will be 2
z = int("3") # z will be 3
```

STRINGS:

SINGLE AND DOUBLE QUOTES:

print("hello,"friend") will not work, and the compiler will throw an error.

Generally, there are two approaches to fixing this. First, you could simply change the quotes to single quotation marks.

Another, more commonly used approach would be code as **print("hello, \"friend\")** -> **escaping technique**. The backslashes tell the compiler that the following character should be

considered a quotation mark in the string

STRING METHODS:

- By utilizing the method `strip` on name as **`name = name.strip()`**, will strip all the whitespaces on the left and right of the users input.
- Using the **title method**, it would title case the user's name
- Using **`first,last=name.split("")`** split string into two and assign them to variables
- Using **"string with {} and {}".format(value1, value2)** positional arguments
- Using **"string with {name} and {age}".format(name="Alice", age=30)** named arguments
- Using **`print("Repeat this {0} {0}!".format("word"))`** reusing arguments
- Using **"The number is {:.2f}".format(3.14159)** formatting numbers
- Using **`age = 30 print(f"You are {age} years old, which is {age * 12} months.")`**
- Using **`print(f"The price is ${price:.2f}")`**
- Using **`f"Hello, {name.upper()}"`**
- Using **`print("hello, " + name)`**
- Using **`print("hello,", name)`**
- Using **`sentence.replace(word, word.upper())`** to replace a word in string
- Using **`return len(word)`** to return length of word
- Using **`letter.isalpha()`** to check if string consists of only letters
- **`"Mountains".upper()`**
- **`"Mountains".lower()`**
- **`"The number of times e occurs in this string is 4".count("e")`**
- **`"Forest".endswith("rest")`**
- **`"Forest".isnumeric()`**
- **`"12345".isnumeric()`**
- **`"...".join(["This", "is", "a", "phrase", "joined", "by", "triple", "dots"])`**
- **`"This is another example".split()`**
- **`print(f'Length: {len(name)}')`** # Length method (includes spaces)
- **`print(f'B occurrence: {name.find('b')})'`** # Find a character in string (returns -1 if not found)
- **`print(f'Last occurrence of 'a': {name.rfind('a')})'`** # Last occurrence of a character
- **`print(name.capitalize())`** # Capitalize first letter
- **`print(name.upper())`** # Upper method (convert to uppercase)
- **`print(name.isdigit())`** # `isdigit()` returns True only if the string consists of only numbers
- **`print(name.isalpha())`** # `isalpha()` returns True only if the string has only alphabets (excluding spaces)
- **`print(name.count('a'))`** # `count()` method counts how many times a character appears in a string
- **`print(name.replace(' ', '-'))`** # `replace()` method replaces spaces with hyphens
- **`print(help(str))`** # If you want a list of string methods

```
def remove_vowels(text):
    result = ""
    for vowel in text:
        if vowel not in ['a','e','i','o','u','A','E','I','O','U']:
            result += vowel
    return result

def main():
    user_input = input("Enter a string of text: ")
    no_vowels_text = remove_vowels(user_input)
    print("Text without vowels:", no_vowels_text)

if __name__ == "__main__":
    main()
```

CHANGING STRINGS:

Fixing a typo in a string using slicing

```
message = "A kong string with a silly typo"
new_message = message[0:2] + "l" + message[3:] # Replacing "k" with "l"
print(new_message) # Output: "A long string with a silly typo"
```

Function to replace an old domain with a new one in an email

```
def replace_domain(email, old_domain, new_domain):
    # Check if the email contains the old domain
    if "@" + old_domain in email:
        index = email.index("@" + old_domain) # Find where the old domain starts
        new_email = email[:index] + "@" + new_domain # Replace it with the new domain
        return new_email # Return the updated email
    return email # If no change is needed, return the original email
```

Example usage:

```
old_email = "user@example.com"
updated_email = replace_domain(old_email, "example.com", "newdomain.com")
print(updated_email) # Output: "user@newdomain.com"
```

STRING INDEXING CHEAT SHEET:

allows to access elements using [] brackets [start:end:step]
 num="1234-5678-9012-3456"
 #[start:end] end is exclusive
 print(num[0:4])
 print(num[:4]) #python assumes start as beginning
 print(num[5:])#python prints from start to end
 print(num[-1])
 print(num[::2])#every 2nd character
 print(num[::-1])#reverse

FINDING INDEX:

```
pets="Cats & Dogs"
pets.index("&")
```

```
pets.index("C")
```

FINDING SUBSTRINGS:

```
pets="Cats & Dogs"
"Dragons" in pets
"Cats" in pets
```

HOW TO SLICE STRINGS:

- `string[start:end]` → Extracts characters from index start to end-1.
- `string[:end]` → Starts from the beginning up to end-1.
- `string[start:]` → Starts from start to the end.
- Negative indexes access characters from the end.
- Slicing includes the start index but excludes the end index.
- Omitting start (`[:end]`) starts from the beginning.
- Omitting end (`[start:]`) continues to the end.
- Negative indexes count from the end.

```
string1 = "Greetings, Earthlings"
print(string1[0]) # Output: "G"
print(string1[4:8]) # Output: "ting" (Includes index 4, excludes index 8)
print(string1[11:]) # Output: "Earthlings"
print(string1[:5]) # Output: "Greet"
```

```
color = "Orange"
print(color[1:4]) # Output: "ran"
```

```
fruit = "Pineapple"
print(fruit[:5]) # Output: "Pinea"
print(fruit[5:]) # Output: "pple"
```

HOW TO JOIN STRINGS :

```
print("Hello" + " " + "world") #Prints "Hello world"
```

You can also use the **join()** function, which is very useful when you want to concatenate elements from a list of strings with a specific delimiter. In the following example, we have a list of strings called greetings and we join them with a space using **.join(greetings)**

```
greetings = ["Hello", "world"]
print(" ".join(greetings)) # Prints "Hello world"
```

INT:

```
x = input("What's x? ")
y = input("What's y? ")
z = x + y
print(z)
x = input("What's x? ")
```

```

y = input("What's y? ")
z = int(x) + int(y)
print(z)
x = int(input("What's x? "))
y = int(input("What's y? "))
print(x + y)

```

FLOAT:

ROUND FUNCTION:

Documentation: Round (number[,ndigits])

Basic Rounding

```

x = float(input("What's x? "))
y = float(input("What's y? "))
z = round(x + y)
print(z) # Rounds sum to nearest integer

```

Formatting Large Numbers with Commas

```

x = float(input("What's x? "))
y = float(input("What's y? "))
z = round(x + y)
print(f'{z:,}') # Outputs with commas (e.g., 1,000)

```

Rounding to Two Decimal Places

```

x = float(input("What's x? "))
y = float(input("What's y? "))
z = round(x / y, 2)
print(z) # Rounds division result to 2 decimal places

```

Using f-string for Decimal Formatting

```

x = float(input("What's x? "))
y = float(input("What's y? "))
z = x / y
print(f'{z:.2f}') # Always shows 2 decimal places (e.g., 5.00)

```

NAMING RULES AND CONVENTIONS:

When assigning names to objects, programmers adhere to a set of rules and conventions which help to standardize code and make it more accessible to everyone. Here are some naming rules and conventions that you should know:

- Names cannot contain spaces.
- Names may be a mixture of upper and lower case characters.
- Names can't start with a number but may contain numbers after the first character.
- Variable names and function names should be written in snake_case, which means that all letters are lowercase and words are separated using an underscore.
- Descriptive names are better than cryptic abbreviations because they help other programmers

(and you) read and interpret your code. For example, `student_name` is better than `sn`. It may feel excessive when you write it, but when you return to your code you'll find it much easier to understand.

VARIABLES ANNOTATED BY TYPE:

Type annotations are optional in Python. They can be very helpful, though, because they make code easier to read. Annotations make the variable types clear to those reading the code. They can also help you catch errors during compilation. In the example below, we are using the typing module to annotate the different types of variables.

```
import typing
# Define a variable of type str
z: str = "Hello, world!"
# Define a variable of type int
x: int = 10
# Define a variable of type float
y: float = 1.23
# Define a variable of type list
list_of_numbers: typing.List[int] = [1, 2, 3]
# Define a variable of type tuple
tuple_of_numbers: typing.Tuple[int, int, int] = (1, 2, 3)
# Define a variable of type dict
dictionary: typing.Dict[str, int] = {"key1": 1, "key2": 2}
# Define a variable of type set
set_of_numbers: typing.Set[int] = {1, 2, 3}
```

UNICODE:

Uppercase		Uppercase		Lowercase		Lowercase	
Unicode #	Character	Unicode #	Character	Unicode #	Character	Unicode #	Character
65	A	78	N	97	a	110	n
66	B	79	O	98	b	111	o
67	C	80	P	99	c	112	p
68	D	81	Q	100	d	113	q
69	E	82	R	101	e	114	r
70	F	83	S	102	f	115	s
71	G	84	T	103	g	116	t
72	H	85	U	104	h	117	u
73	I	86	V	105	i	118	v
74	J	87	W	106	j	119	w
75	K	88	X	107	k	120	x
76	L	89	Y	108	l	121	y
77	M	90	Z	109	m	122	z

FUNCTIONS:

USER-DEFINED FUNCTIONS:

A function is a piece of code that performs a unit of work. In the examples you've seen so far, you have only encountered the **print()** function, which outputs a message to the screen. You will use this function frequently in this course to check the results of your code. The syntax of the **print()** function is modeled in the example below.

Documentation: `Print(*objects,sep=',',end='\n')`

Basic Function

```
def hello():
    print("hello")

name = input("What's your name? ")
hello()
print(name) # Calls hello() and prints name
```

Function with Argument

```
def hello(to):
    print("hello,", to)

name = input("What's your name? ")
hello(name) # Passes user input to function
```

Function with Default Argument

```
def hello(to="world"):
    print("hello,", to)

def main():
    name = input("What's your name? ")
    hello(name) # Calls hello with input
    hello() # Calls hello with default "world"
```

```
main()
```

Function Returning a Value

```
def main():
    x = int(input("What's x? "))
    print("x squared is", square(x))

def square(n):
    return n * n # Returns square of n

main()
```

BUILT-IN FUNCTIONS:**print()**

print(): Outputs a specified object to the screen

```
print("Hello, world!")
```

type():

Returns the data type of a variable

```
x = "Hello"
```

```
print(type(x)) # Output: <class 'str'>
```

sorted():

Sorts elements in a list (without modifying the original list)

```
time_list = [12, 2, 32, 19, 57, 22, 14]
```

```
print(sorted(time_list)) # Output: [2, 12, 14, 19, 22, 32, 57]
```

```
print(time_list) # Original list remains unchanged
```

max() and min():

Find the largest and smallest values in a list

```
print(min(time_list)) # Output: 2
```

```
print(max(time_list)) # Output: 57
```

CONDITIONALS:**COMPARISON OPERATORS WITH EQUATIONS:**

- == (equality)
- != (not equal to)
- > (greater than)
- < (less than)
- >= (greater than or equal to)

- <= (less than or equal to)

LOGICAL OPERATORS:

and: Both sides of the statement must be True for the whole statement to be True.

or: If either side of the comparison is True, then the whole statement is True.

not: Inverts the Boolean result of the statement that follows it.

IF/ELSE/ELIF:

Username Validation

```
if len(username) < 3:
    print("Invalid username. Must be at least 3 characters long")
else:
    print("Valid username")
```

Function for Username Hints

```
def hint_username(username):
    if len(username) < 3:
        print("Invalid username. Must be at least 3 characters long")
    elif len(username) > 15:
        print("Invalid username. Must be at most 15 characters long")
    else:
        print("Valid username")
```

Check if a Number is Even or Odd

```
def main():
    x = int(input("What's x? "))
    print("Even" if is_even(x) else "Odd")
```

```
def is_even(n):
    return n % 2 == 0 # Returns True if even, False otherwise
```

```
main()
```

House Sorting

```
name = input("What's your name? ")
if name in ["Harry", "Hermione", "Ron"]:
    print("Gryffindor")
elif name == "Draco":
    print("Slytherin")
else:
    print("Who?")
```

Weight Converter

```
weight = float(input("Enter your weight: "))
choice = input("kg or pounds? ")
if choice == "pounds":
    print(f"Your weight {weight}kg is {round(weight * 2.2, 1)} pounds")
```

```
elif choice == "kg":
    print(f"Your weight {weight} pounds is {round(weight / 2.2, 1)} kg")
```

TERNARY OPERATOR:

shortcut of if-else statement

X if condition else Y

```
num=5
```

```
a=6
```

```
b=7
```

```
print("Positive" if num>0 else "Negative")
```

```
max_num=a if a>b else b
```

```
print (max_num)
```

MATCH STATEMENT:

Using match-case for House Sorting

```
name = input("What's your name? ")
```

```
match name:
```

```
    case "Harry":
```

```
        print("Gryffindor")
```

```
    case "Hermione":
```

```
        print("Gryffindor")
```

```
    case "Ron":
```

```
        print("Gryffindor")
```

```
    case "Draco":
```

```
        print("Slytherin")
```

```
    case _:
```

```
        print("Who?")
```

Optimized match-case with multiple conditions

```
match name:
```

```
    case "Harry" | "Hermione" | "Ron":
```

```
        print("Gryffindor")
```

```
    case "Draco":
```

```
        print("Slytherin")
```

```
    case _:
```

```
        print("Who?")
```

LOOPS:

WHILE LOOP:

A **while** loop executes the body of the loop while a specified condition remains **True**. They are commonly used when there's an unknown number of operations to be performed, and a condition needs to be checked at each iteration.

While Loop Example

```
x = 0
```

```
while x < 5:
```

```
    print("Not there yet, x=" + str(x))
```

```
    x += 1
```

```
print("x=" + str(x))
```

Valid Username Check

```
username = get_username()
while not valid_username(username):
    print("Invalid username")
    username = get_username()
```

Multiplication Loop

```
multiplier = 1
result = multiplier * 5
while result <= 50:
    print(result)
    multiplier += 1
    result = multiplier * 5
print("Done")
```

Function to Count Factors of a Given Number

```
def count_factors(given_number):
    if given_number == 0:
        return 0 # Zero has no factors

    factor = 1
    count = 1 # Includes the given_number itself

    while factor < given_number:
        if given_number % factor == 0:
            count += 1
        factor += 1

    return count

# Testing the count_factors function
print(count_factors(0)) # Output: 0
print(count_factors(3)) # Output: 2 (1x3)
print(count_factors(10)) # Output: 4 (1x10, 2x5)
print(count_factors(24)) # Output: 8 (1x24, 2x12, 3x8, 4x6)
```

```
food=input("Enter q to quit:")
```

```
#method1
```

```
while not food=='q':
    print(food)
    break
```

```
#method 2
```

```
while not food=='q':
    print(food)
    food=input("enter another food")
```

FOR LOOP:**Start (Default: 0)**

- The first parameter defines the starting position of the range.

- If not provided, it defaults to 0.

Stop (Required)

- The second parameter defines the stopping position (exclusive).
- Must be specified because there is no default.

Step (Default: +1)

- The third parameter defines the step size (increment).
- Default is +1, but can be changed to any valid value

Reverse Counting

```
for x in reversed(range(1, 11)): print(x)
for x in range(11, 1, -1): print(x) # Another way to reverse
```

Print "meow" multiple times

```
print("meow\n" * 3, end="")
```

```
def main():
    meow(get_number())
```

```
def get_number():
    while True:
        n = int(input("What's n? "))
        if n > 0: return n
```

```
def meow(n):
    for _ in range(n): print("meow")
```

```
main()
```

Sum & Average Calculation

```
values = [23, 52, 59, 37, 48]
sum, length = 0, 0
for value in values: sum += value; length += 1
print(f"Total sum: {sum} - Average: {sum/length}")
```

Factorial Calculation

```
product = 1
for n in range(1, 10): product *= n
print(product)
```

Fahrenheit to Celsius Conversion

```
def to_celsius(x): return (x - 32) * 5 / 9
for x in range(0, 101, 10): print(x, to_celsius(x))
```

List Comprehension - Squaring Numbers

```
numbers = [1, 2, 3, 4, 5]
squared_numbers = [x ** 2 for x in numbers]
print(squared_numbers)
```

Printing Patterns

```
def main():
    print_row(4)
    print_column(3)
    print_square(3)
```

```
def print_row(width): print("?" * width)
```

```
def print_column(height):
    for _ in range(height): print("#")

def print_square(size):
    for i in range(size):
        for j in range(size): print("#", end="")
        print()
main()
```

RECURSION:

A recursive function calls itself usually with a modified parameter until it reaches a specific condition. This condition is called the base case.

def factorial(n):

```
print("Factorial called with " + str(n))
if n < 2:
    print("Returning 1")
    return 1
result = n * factorial(n-1)
print("Returning " + str(result) + " for factorial of " + str(n))
return result
factorial(4)
```

COLLECTION:

➤ **List ([])**

- Ordered and changeable (mutable).
- Allows duplicate values.
- Uses indexing to access elements.
- Supports various methods like .append(), .remove(), .sort(), etc.
- Example: my_list = [1, 2, 3, 4, 2]

➤ **Set ({})**

- Unordered and immutable (cannot change elements but can add/remove).
- Does not allow duplicate values.
- Elements are unique.
- Supports .add(), .remove(), and set operations (union, intersection, etc.).
- Example: my_set = {1, 2, 3, 4}

➤ **Tuple (())**

- Ordered and unchangeable (immutable).
- Allows duplicate values.
- Faster than lists due to immutability.

- Supports indexing but no modification after creation.
- Example: `my_tuple = (1, 2, 3, 4, 2)`

➤ **Dictionary** ({key: value})

- Stores key-value pairs.
- Ordered (Python 3.7+), changeable, and does not allow duplicate keys.
- Keys must be unique and immutable (strings, numbers, tuples).
- Supports methods like `.get()`, `.keys()`, `.values()`, `.items()`, `.pop()`.
- Example: `my_dict = {"name": "Alice", "age": 25, "city": "New York"}`

LISTS []:

LISTS METHODS:

- **list[index] = x**- Replaces the element at index [n] with x.
- **list.append(x)**- Appends x to the end of the list.
- **list.insert(index, x)**- Inserts x at index position [index].
- **list.pop(index)**- Returns the element at [index] and removes it from the list. If [index] position is not in the list, the last element in the list is returned and removed.
- **list.remove(x)**- Removes the first occurrence of x in the list.
- **list.sort()**- Sorts the items in the list.
- **List.sorted(x, key)**- Sorts according to key and returns a new list
- **list.reverse()**- Reverses the order of items of the list.
- **list.clear()**- Deletes all items in the list.
- **list.copy()**- Creates a copy of the list.
- **list.extend(other_list)**- Appends all the elements of other_list at the end of list
- **map(function, iterable)**- Applies a given function to each item of an iterable (such as a list) and returns a map object with the results
- **zip(*iterables)**- Takes in iterables as arguments and returns an iterator that generates tuples, where the i-th tuple contains the i-th element from each of the argument iterables.
- **print(dir(fruits))** # Display available methods for the list
- **print(help(fruits))** # Display help documentation for list methods
- **print(len(fruits))** # Get the length of the list
- **print("apple" in fruits)** # Check if "apple" is in the list
- **fruits[0] = "pineapple"** # Modify the first element of the list
- **fruits.append("pineapple")** # Append "pineapple" to the end of the list
- **fruits.remove("apple")** # Remove the first occurrence of "apple" from the list
- **fruits.insert(0, "pineapple")** # Insert "pineapple" at index 0
- **fruits.sort()** # Sort the list in ascending order
- **fruits.reverse()** # Reverse the order of the list
- **fruits.clear()** # Clear all elements from the list
- **fruits.index("apple")** # Get the index of "apple" in the list
- **fruits.count("apple")** # Count occurrences of "apple" in the list

```
students = ["Hermoine", "Harry", "Ron"]
```

```
for student in students:
```

```
    print(student)
```

```
students = ["Hermoine", "Harry", "Ron"]
```

```
for i in range(len(students)):
    print(i + 1, students[i])
```

OUTPUT:

```
1 Hermoine
2 Harry
3 Ron
```

MODIFYING LISTS:

```
fruits = ["Pineapple", "Banana", "Apple", "Melon"]
fruits.append("Kiwi")
print(fruits)
fruits = ["Pineapple", "Banana", "Apple", "Melon"]
fruits.insert(0, "Orange")
fruits.insert(25, "Peach")
fruits.remove("Melon")
print(fruits)
fruits = ["Pineapple", "Banana", "Apple", "Melon"]
fruits.insert(0, "Orange")
fruits.insert(25, "Peach")
fruits.remove("Melon")
fruits.pop(3)
fruits[2] = "Strawberry"
print(fruits)
```

ITERATING OVER LISTS AND TUPLES:

Using enumerate() to print index and name:

```
winners = ["Ashley", "Dylan", "Reese"]
for index, person in enumerate(winners):
    print("{} - {}".format(index + 1, person))
```

```
# Output:
# 1 - Ashley
# 2 - Dylan
# 3 - Reese
```

Function to format emails

```
def full_emails(people):
    result = []
    for email, name in people:
        result.append("{} <{}>".format(name, email))
    return result
```

```
print(full_emails([
    ("alex@example.com", "Alex Diego"),
    ("shay@example.com", "Shay Brandt")
]))
```

Calculating total and average length of animal names

```
animals = ["Lion", "Zebra", "Dolphin", "Monkey"]
chars = 0
```

```
for animal in animals:
    chars += len(animal)
```

```
print("Total characters: {}, Average length: {}".format(chars, chars / len(animals)))
```

Function to print a biography list

```
def biography_list(people):
    # Iterate over each "person" in the given "people" list of tuples.
    for name, age, profession in people:
        print("{} is {} years old and works as {}".format(name, age, profession))
```

Call to the function

```
biography_list([
    ("Ira", 30, "a Chef"),
    ("Raj", 35, "a Lawyer"),
    ("Maria", 25, "an Engineer")
])
```

LIST COMPREHENSIONS:

- concise way to create lists
- **Syntax:**[expression for value in iterable if condition]

List comprehension to create a list of numbers doubled from 1 to 10

```
double = [x * 2 for x in range(1, 11)]
```

Convert all fruit names to uppercase using list comprehension

```
fruits = ["apple", "oranges"]
fruits = [fruit.upper() for fruit in fruits]
```

Create a new list containing only positive numbers

```
number = [1, -2, 4, -5]
pos_num = [num for num in number if num >= 0]
```

Generate multiples of 7 using a loop

```
multiples = []
for x in range(1, 11):
    multiples.append(x * 7)
```

```
print(multiples)
```

Using list comprehension for the same task

```
multiples = [x * 7 for x in range(1, 11)]
print(multiples)
```

Calculate the lengths of programming language names

```
languages = ["Python", "Perl", "Ruby", "Go", "Java", "C"]
lengths = [len(language) for language in languages]
print(lengths)
```

Create a list of numbers divisible by 3 from 0 to 100

```
z = [x for x in range(0, 101) if x % 3 == 0]
print(z)
```

File name modifications

```
filenames = ["program.c", "stdio.hpp", "sample.hpp", "a.out", "math.hpp", "hpp.out"]
```

Replace "hpp" with "h" if filename ends with "hpp"

```
new_filenames = [filename.replace("hpp", "h") if filename.endswith("hpp") else filename for
filename in filenames]
print(new_filenames)
```

Function to convert text into Pig Latin

```
def pig_latin(text):
    say = []
    words = text.split() # Splits the text into words
    for word in words:
        pig_word = word[1:] + word[0] + "ay" # Move the first letter to the end and add "ay"
        say.append(pig_word)
    return " ".join(say) # Join words back into a sentence
```

```
print(pig_latin("hello how are you")) # Expected: "ellohay owhay reaaay ouyay"
print(pig_latin("programming in python is fun")) # Expected: "rogrammingpay niay ythonpay siay unfay"
```

Function to double a number

```
def do_something(x):
    return x * 2
```

Applying the function to a list

```
list_of_things = [1, 2, 3]
new_list = [do_something(thing) for thing in list_of_things] # Using list comprehension
print(new_list)
```

Function to return multiple calculations

```
def calculate_numbers(a, b):
    return a + b, a - b, a * b, a / b
```

Get all results as a tuple

```
result = calculate_numbers(10, 2)
```

```
print(result) # Outputs: (12, 8, 20, 5.0)
```

Unpacking the results

```
add_result, sub_result, mul_result, div_result = calculate_numbers(10, 2)
print(add_result) # 12
print(sub_result) # 8
```

2D LISTS:

Lists within a list used for matrices, grids, etc.

```
fruits = ["apple", "orange", "banana", "coconut"]
vegetables = ["celery", "carrots", "potatoes"]
meats = ["chicken", "fish", "turkey"]
```

2D list (list of lists)

```
groceries = [fruits, vegetables, meats]
```

Accessing 2D list

```
print(groceries[0]) # Prints the whole fruits list
print(groceries[1]) # Prints the whole vegetables list
print(groceries[2]) # Prints the whole meats list
```

Accessing specific elements in the list

```
print(groceries[0][0]) # Prints "apple"
```

Loop through the 2D list

```
for collection in groceries:
    for food in collection:
        print(food, end=" ") # Print items in the same line
    print() # New line after each category
```

SETS {}:

SETS METHODS:

- **fruits = {"apple", "orange", "banana", "coconut"}** → Creates a set of fruits.
- **print(fruits)** → Prints the set (unordered).
- **print(dir(fruits))** → Displays available set methods.
- **print(help(fruits))** → Shows documentation for set methods.
- **print(len(fruits))** → Returns the number of elements in the set.
- **print("apple" in fruits)** → Checks if "apple" is in the set.
- **fruits.add("pineapple")** → Adds "pineapple" to the set.
- **fruits.remove("apple")** → Removes "apple" from the set (error if not found).
- **fruits.pop()** → Removes and returns a random element.
- **fruits.clear()** → Empties the set.
- **print(fruits)** → Prints the now empty set.

TUPLES ():

TUPLES METHODS:

- **print(dir(fruits))** → Displays available methods for tuples.
- **print(help(fruits))** → Shows documentation for tuple methods.
- **print(len(fruits))** → Returns the number of elements in the tuple.
- **print("apple" in fruits)** → Checks if "apple" exists in the tuple.

- **fruits.index("apple")** → Returns the index of the first occurrence of "apple" in the tuple.
- **fruits.count("apple")** → Returns the number of times "apple" appears in the tuple.
- **for fruit in fruits: print(fruit)** → Iterates through the tuple and prints each element.
- **Note:** Since tuples are **immutable**, you **cannot** modify them (e.g., adding or removing elements).

DICT ({key: value}):

dicts or dictionaries is a data structure that allows you to associate keys with values.

Where a list is a list of multiple values, a dict associates a key with a value.

DICT METHODS:

- **len(dictionary)** - Returns the number of items in a dictionary.
- **for key, in dictionary** - Iterates over each key in a dictionary.
- **for key, value in dictionary.items()** - Iterates over each key,value pair in a dictionary.
- **if key in dictionary** - Checks whether a key is in a dictionary.
- **dictionary[key]** - Accesses a value using the associated key from a dictionary.
- **dictionary[key] = value** - Sets a value associated with a key.
- **del dictionary[key]** - Removes a value using the associated key from a dictionary.
- **dictionary.get(key, default)** - Returns the value corresponding to a key, or the default value if the specified key is not present.
- **dictionary.keys()** - Returns a sequence containing the keys in a dictionary.
- **dictionary.values()** - Returns a sequence containing the values in a dictionary.
- **dictionary[key].append(value)** - Appends a new value for an existing key.
- **dictionary.update(other_dictionary)** - Updates a dictionary with the items from another dictionary. Existing entries are updated; new entries are added.
- **dictionary.clear()** - Deletes all items from a dictionary.
- **dictionary.copy()** - Makes a copy of a dictionary.
- **get(key):** Retrieves the value associated with a key.
- **update({key: value}):** Adds or updates key-value pairs.
- **pop(key):** Removes a specific key-value pair.
- **popitem():** Removes the last inserted key-value pair.
- **clear():** Removes all entries.
- **keys():** Returns all keys.
- **values():** Returns all values.
- **items():** Returns all key-value pairs.
- **for key, value in capitals.items():** Loops through dictionary items.

#collection of {key:value}

```
capitals={"USA":"Washington","India":"New Delhi","China":"Beijing"}
```

#get key value:

```
print(capitals.get("USA"))
```

```
#if capitals.get():print("exists")
```

#update method:

```
capitals.update({"germany":"Berlin"})
```

```
capitals.update({"USA":"Berlin"})
```

```

#remove a key value:
capitals.pop("China")
capitals.popitem() #will remove the latest key
capitals.clear()
#keys:
keys=capitals.keys()
print(keys)
#iterable:
for key in capitals.keys():
    print(key)
#values method:
values=capitals.values()
print(values)
#Items method:
items=capitals.items()
print(items)
for key,value in capitals.items():
    print(f"{key}:{value}")

```

Dictionary Example

```

file_counts = {"jpg": 10, "txt": 14, "csv": 2, "py": 23}
print(file_counts)

```

Accessing a value

```

print(file_counts["txt"])

```

Adding a new key-value pair

```

file_counts["cfg"] = 8
print(file_counts)

```

Replacing a value

```

file_counts["csv"] = 17
print(file_counts)

```

Deleting a key-value pair

```

del file_counts["cfg"]
print(file_counts)

```

Printing keys and values

```

for extension in file_counts:
    print(extension, file_counts[extension])

```

```

for ext, amount in file_counts.items():
    print(f"There are {amount} files with the .{ext} extension")

```

Accessing keys and values

```

print(file_counts.keys())
print(file_counts.values())

```

Counting letter occurrences in a string

```

def count_letters(text):
    result = {}
    for letter in text:
        if letter not in result:
            result[letter] = 0
        result[letter] += 1

```

```

    return result

print(count_letters("aaaaa"))
print(count_letters("tenant"))
print(count_letters("a long string with a lot of letters"))

# Dictionary with student houses
students = {
    "Hermione": "Gryffindor",
    "Harry": "Gryffindor",
    "Ron": "Gryffindor",
    "Draco": "Slytherin"
}

print(students["Hermione"])
print(students["Harry"])
print(students["Ron"])
print(students["Draco"])

# Iterating through a dictionary
for student in students:
    print(student, students[student])

# List of dictionaries with student details
students = [
    {"name": "Hermione", "house": "Gryffindor", "patronus": "Otter"},
    {"name": "Harry", "house": "Gryffindor", "patronus": "Stag"},
    {"name": "Ron", "house": "Gryffindor", "patronus": "Jack Russell terrier"},
    {"name": "Draco", "house": "Slytherin", "patronus": None},
]

for student in students:
    print(student["name"], student["house"], student["patronus"], sep=", ")

# Function to generate email list
def email_list(domains):
    emails = []
    for domain, users in domains.items():
        for user in users:
            email = f"{user}@{domain}"
            emails.append(email)
    return emails

print(email_list({
    "gmail.com": ["clark.kent", "diana.prince", "peter.parker"],
    "yahoo.com": ["barbara.gordon", "bruce.wayne"]
}))

# Function to group users by their groups
def groups_per_user(group_dictionary):
    user_groups = {}

    # Go through group_dictionary
    for group, users in group_dictionary.items():
        # Now go through the users in the group
        for user in users:
            # Now add the group to the list of groups for this user,
            # creating the entry in the dictionary if necessary

```

```

        if user not in user_groups:
            user_groups[user] = []
            user_groups[user].append(group)

    return user_groups

print(groups_per_user({
    "local": ["admin", "userA"],
    "public": ["admin", "userB"],
    "administrator": ["admin"]
}))

```

OOP:

In Python, methods are behaviors associated with object parameters that modify the state of that object. They are essentially functions that belong to a specific instance of a class. This means that calling a method on a list, for example, only modifies that instance of the list, and not all lists globally.

Object=attributes(characteristics)+Methods

```

class Car:
    #constructor to create objects
    #dunder method
    #self means "this" object(Car) object we are currently working with
    #attributes:
    def __init__(self,model,year,color,for_sale):
        self.model=model
        self.year=year
        self.color=color
        self.for_sale=for_sale
    #methods:
    def drive(self):
        print("You drive a car")
#object creation:
car1=Car("Mustang","2024","red",False)
print(car1.model)
print(car1.drive())

```

INSTANCE VS CLASS VARIABLES:

Instance Variables: Defined inside the `__init__` constructor. Each object gets its own copy of these variables.

Class Variables: Defined outside the constructor. Shared by all instances of the class.

```

class Student:
    class_year = 2024 # Class variable
    num_students = 0 # Class variable to count number of students

    def __init__(self, name, age):
        self.name = name # Instance variable
        self.age = age # Instance variable
        Student.num_students += 1 # Increment student count

```

Creating instances

```
s1 = Student("Taiba", 20)
s2 = Student("Humna", 18)
```

Accessing instance variables

```
print(s1.name)
print(s1.age)
```

Accessing class variable through instance

```
print(s1.class_year)
```

Accessing class variable through class itself (preferred way)

```
print(Student.class_year)
```

SINGLE-INHERITANCE:

Parent Class

```
class Animal:
    def __init__(self, name):
        self.name = name
        self.is_alive = True # Default value set in constructor

    def eat(self):
        print(f'{self.name} is eating')
```

Child Class (inherits from Animal)

```
class Dog(Animal):
    def speak(self):
        print("Woof!")
```

Another Child Class

```
class Cat(Animal):
    def speak(self):
        print("Meow!") # Corrected to "Meow" instead of "Woof"
```

Creating objects of child classes

```
dog = Dog("Scooby")
cat = Cat("Garfield")
```

Accessing inherited attributes and methods

```
print(dog.name) # Scooby
dog.eat()       # Scooby is eating
dog.speak()     # Woof!
```

```
print(cat.name) # Garfield
cat.eat()       # Garfield is eating
cat.speak()     # Meow!
```

MULTIPLE & MULTI-LEVEL INHERITANCE:**MULTIPLE:** A child class inherits from multiple parent classes. $C(A,B)$.**MULTI-LEVEL:** A child class inherits from another child class. $C(B) \leftarrow B(A) \leftarrow A$.**# Parent class**

```
class Animal:
    def eat(self):
        print("This animal is eating")
```

Multi-level Inheritance: Prey inherits from Animal

```
class Prey(Animal):
    def flee(self):
        print("This animal is fleeing")
```

Predator also inherits from Animal

```
class Predator(Animal):
    def hunt(self):
        print("This animal is hunting")
```

Rabbit inherits from Prey (Multi-level Inheritance)

```
class Rabbit(Prey):
    pass
```

Hawk inherits from Predator

```
class Hawk(Predator):
    pass
```

Fish inherits from both Prey and Predator (Multiple Inheritance)

```
class Fish(Prey, Predator):
    pass
```

Object creation

```
rabbit = Rabbit()
hawk = Hawk()
fish = Fish()
```

Testing methods

```
rabbit.flee() # From Prey
hawk.hunt() # From Predator
fish.hunt() # From Predator
fish.flee() # From Prey
rabbit.eat() # Inherited from Animal
```

POLYMRPHISM:Polymorphism means "**many forms**", where:**Inheritance-based Polymorphism** → Different child classes override the same method from a parent class.**Duck Typing** → If an object has the required method, it can be used interchangeably (regardless of its class).**# INHERITANCE:**

An object can be treated as the same type as its parent class

from abc import ABC, abstractmethod

```
class Shape(ABC):
```

```
    # Abstract method is initialized in base class but defined in inherited class
```

```

@abstractmethod
def area(self):
    pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius**2

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side**2

class Triangle(Shape):
    def __init__(self, base, height):
        self.base = base
        self.height = height

    def area(self):
        return 0.5 * self.base * self.height

# A pizza is a pizza, but it is circular:
# Pizza -> Circle -> Shape (Different forms of a pizza)
class Pizza(Circle):
    def __init__(self, toppings, radius):
        self.toppings = toppings
        super().__init__(radius)

# List of different shapes
shapes = [Circle(4), Square(5), Triangle(6, 7), Pizza("Pepperoni", 15)]

for shape in shapes:
    print(f"{shape.area()} cm^2")

# DUCK TYPING:
# In Duck Typing, an object does not need to inherit from a common parent class.
# It just needs to have the necessary attributes or methods.

class Animal:
    alive = True

class Dog(Animal):
    def speak(self):
        print("Woof")

class Cat(Animal):
    def speak(self):
        print("Meow")

# Completely unrelated class, but it also has a `speak()` method.

```

This means it can be used in the same way as Dog and Cat.

```
class Car:
    alive = False

    def speak(self):
        print("Honk")
```

List of different objects that all have a `speak()` method

```
animals = [Dog(), Cat(), Car()]

for animal in animals:
    animal.speak() # Calls speak() method, no matter the class
    print(animal.alive) # Checks if the object is "alive"
```

METHODS :

INSTANCE METHODS:

Instance methods are the most commonly used methods in Python. They are associated with a specific instance of the class, meaning they can access and modify instance-specific data using the self parameter. Operate on object instances and have access to self

```
class Employee:

    def __init__(self, name, position) -> None:
        self.name = name
        self.position = position

    # Instance method -> operates on individual instances
    def get_info(self):
        return f"{self.name} = {self.position}"

# Accessing instance methods using an object
employee1 = Employee("Eugene", "Manager")
print(employee1.get_info()) # Eugene = Manager
```

CLASS METHODS:

Class methods, on the other hand, are called for the class itself instead of an instance. They are marked with a **@classmethod** decorator and take a **cls** parameter that points to the class—and not any specific instance—when the method is called. One common use-case for class methods is to create and modify data structures that contain records for all instances of a class. Usually, programmers make a list inside the class definition, and methods to add instances of the class to that list in order to keep track of that class.

- allow operations related to class itself

- take(cls) as first parameter -> class itself

class Dog:

```
all_dogs = [] # Class variable to store all dog instances

def __init__(self, name): # Constructor method
    self.name = name
    Dog.all_dogs.append(self) # Add new dog instance to the list

@classmethod
def count_dogs(cls): # Class method to count all dog instances
    return len(cls.all_dogs)
```

Create instances

```
dog1 = Dog("Buddy")
dog2 = Dog("Max")
```

Call class method

```
print(Dog.count_dogs()) # Output: 2
```

class Student:

```
# Class variables (shared by all instances)
count = 0
total_gpa = 0

def __init__(self, name, gpa):
    self.name = name
    self.gpa = gpa
    Student.count += 1
    Student.total_gpa += gpa
```

Instance method

```
def get_info(self):
    return f"{self.name} {self.gpa}"
```

Class method to get student count

```
@classmethod
def get_count(cls):
    return f"Total students: {cls.count}"
```

Class method to calculate average GPA

```
@classmethod
def get_average_gpa(cls):
    if cls.get_count() == "Total students: 0": # Fix condition
        return 0
    else:
        return f"Average GPA: {cls.total_gpa / cls.count}"
```

Calling class methods

```
print(Student.get_count())    # Output: Total students: 0 (Initially)
print(Student.get_average_gpa()) # Output: 0 (No students yet)
```

Creating instances

```
s1 = Student("Alice", 3.8)
s2 = Student("Bob", 3.5)
```

Calling class methods again after adding students

```
print(Student.get_count())    # Output: Total students: 2
print(Student.get_average_gpa()) # Output: Average GPA: 3.65
```

STATIC METHODS:

Lastly, static methods, marked with a **@staticmethod** decorator, do not take a **self** or a **cls** parameter. Static methods behave like plain functions, except that you can call them directly from the class. It is important to note that you do not have to actually instantiate the class, the methods just reside in there. This is because class definitions are themselves an object (i.e., an instance of abstract base class), which reduces overhead and allows functions to be encapsulated in an easy-to-use encapsulation. Programmers use static methods when the method does not need to access any instance or class-specific data.

@staticmethod

```
# Static method -> does not need class instance (self) or class data
def is_valid(position):
    valid_positions = ["Manager", "Cashier", "Cook", "Janitor"]
    return position in valid_positions
```

Accessing static methods using the class name

```
print(Employee.is_valid("Manager")) # ☒ True
```

DUNDER METHODS:

Dunder methods (short for "**double underscore**" methods) are **special built-in methods** in Python that **start and end with double underscores** (`__method__`). They are also called "**magic methods**" because they allow us to define custom behavior for built-in operations. They help define custom behavior of objects.

class Book:

```
def __init__(self, title, author, num_pg):
    # Initializes book attributes
    self.title = title
    self.author = author
    self.num_pg = num_pg
```

Returns a readable string when printing the object

```
def __str__(self):
    return f'{self.title} by {self.author}'
```

```
def __eq__(self, other):
```

Checks if two books have the same title and author

return self.title == other.title and self.author == other.author

def __lt__(self, other):

Compares books based on the number of pages

return self.num_pg < other.num_pg

def __add__(self, other):

Adds two books by summing their page counts

return self.num_pg + other.num_pg

def __contains__(self, keyword):

Checks if a keyword exists in the book title

return keyword in self.title

def __getitem__(self, key):

Allows accessing book attributes like a dictionary

if key == "title":

return self.title

Creating book objects

book1 = Book("The Hobbit", "J.R.R. Tolkien", 310)

book4 = Book("The Hobbit", "J.R.R. Tolkien", 320)

book2 = Book("Harry Potter", "J.K. Rowling", 223)

book3 = Book("The Lion", "Lewis", 172)

Demonstrating dunder method functionality

print(book1) # Uses __str__

print(book1 == book2) # Uses __eq__

print(book1 == book4) # Checks if books have the same title & author

print(book2 < book3) # Uses __lt__

print(book2 + book3) # Uses __add__

print("Lion" in book3) # Uses __contains__

print(book3["title"]) # Uses __getitem__

CONSTRUCTOR:

When writing a Python class, you define a method called **`__init__`** to be your constructor. The special name tells Python to use that method as the constructor. Just like any other method, the constructor can take arguments. When making an argument to the class, the first constructor must always be **self**.

class Apple:

def __init__(self, color, flavor): # Corrected indentation and syntax

self.color = color

self.flavor = flavor

Creating objects (instances)

honeycrisp = Apple("red", "sweet")

fuji = Apple("red", "tart")

Accessing attributes

```
print(honeycrisp.flavor) # Output: sweet
print(fuji.flavor)      # Output: tart
```

Adding a string representation method

```
class Apple:
    def __init__(self, color, flavor):
        self.color = color
        self.flavor = flavor

    def __str__(self): # Corrected __str__ method
        return "An apple which is {} and {}".format(self.color, self.flavor)
```

Creating an object

```
honeycrisp = Apple("red", "sweet")
print(honeycrisp) # Output: An apple which is red and sweet
```

DECORATORS:

A decorator in Python is a special function that modifies the behavior of another function or method. It allows you to extend functionality without changing the original code. Decorators are applied using the `@decorator_name` syntax above a function. Think of it like a wrapper around a function—it can add extra functionality before or after the function runs.

A decorator is a function that extends the behavior of another function # without modifying its original code.

```
def add_sprinkles(func):
    def wrapper(*args, **kwargs): # Allows any number of arguments
        print("You add sprinkles 🍩")
        func(*args, **kwargs) # Calls the original function
    return wrapper
```

```
def add_fudge(func):
    def wrapper(*args, **kwargs):
        print("You add fudge 🍫")
        func(*args, **kwargs)
    return wrapper
```

@add_sprinkles

@add_fudge

```
def get_ice_cream(flavour):
    print(f'Here is your {flavour} ice cream 🍦')
```

Calling the function

```
get_ice_cream("vanilla")
```

PROPERTY DECORATOR:

The `@property` decorator in Python is used to define a **getter method** that allows you to access a method like an attribute. It is used to manage access to instance variables in an object-oriented way.

- **Getter (@property)** → Allows reading the attribute as if it were public.
- **Setter (@property_name.setter)** → Allows setting a value with validation.
- **Deleter (@property_name.deleter)** → Allows deleting the attribute safely

class Rectangle:

```
def __init__(self, width, height):
    self._width = width # Private attributes
    self._height = height
```

Getter methods

@property

```
def width(self):
    return self._width
```

@property

```
def height(self):
    return self._height
```

Setter methods

@width.setter

```
def width(self, new_width):
    if new_width > 0:
        self._width = new_width
    else:
        print("Invalid width")
```

@height.setter

```
def height(self, new_height):
    if new_height > 0:
        self._height = new_height
    else:
        print("Invalid height")
```

Deleter methods

@width.deleter

```
def width(self):
    print("Width deleted")
    del self._width
```

@height.deleter

```
def height(self):
    print("Height deleted")
    del self._height
```

Example usage

```
rectangle = Rectangle(3, 4)
print(rectangle.width) # 3
print(rectangle.height) # 4
```

```
rectangle.height = 5 # Update height
print(rectangle.height) # 5
```

```
del rectangle.width # Deletes width
# print(rectangle.width) # Uncommenting this will raise an AttributeError
```

FILE HANDLE I/O (.txt .csv.json):

Up until now, everything we've programmed has stored information in memory. That is, once the program is ended, all information gathered from the user or generated by the program is lost. File I/O is the ability of a program to take a file as input or create a file as output. When a program runs, it stores data in memory (RAM), which is **temporary**. As soon as the program ends, all data is **lost** unless we save it somewhere, like in a **file**.

```
names = []

# Collect 3 names from the user
```

```
for _ in range(3):
    name = input("What's your name? ")
    names.append(name)
```

```
# Sort names and print greetings
```

```
for name in sorted(names):
    print(f"Hello, {name}")
```

FILE & DIRECTORY METHODS (OS AND OS.PATH):

◆ **File Handling Methods**

Method	Description
os.path.exists(path)	Checks if a file or directory exists.
os.path.isfile(path)	Checks if the path is a file.
os.path.isdir(path)	Checks if the path is a directory.
os.path.abspath(path)	Returns the absolute path of a file.
os.path.basename(path)	Returns the filename from a path.
os.path.dirname(path)	Returns the directory of a file.
os.path.getsize(path)	Returns the size of a file in bytes.
os.rename(src, dst)	Renames a file or directory.
os.remove(path)	Deletes a file.

Method	Description
<code>os.replace(src, dst)</code>	Renames a file or directory, replacing if it exists.

◆ Directory Handling Methods

Method	Description
<code>os.mkdir(path)</code>	Creates a new directory (folder).
<code>os.makedirs(path)</code>	Creates directories including parent directories if they don't exist.
<code>os.rmdir(path)</code>	Deletes a directory (only if it's empty).
<code>os.removedirs(path)</code>	Deletes a directory and parent directories if they're empty.
<code>os.listdir(path)</code>	Returns a list of files and folders in a directory.
<code>os.getcwd()</code>	Returns the current working directory .
<code>os.chdir(path)</code>	Changes the current working directory .
<code>os.walk(path)</code>	Recursively iterates through directories & files .

◆ File Opening, Reading, Writing (open() Methods)

Mode	Description
"r"	Read Mode – Opens a file for reading (default).
"w"	Write Mode – Creates a file (or overwrites if exists).
"a"	Append Mode – Adds data to a file without overwriting .
"rb" / "wb"	Opens a file in binary mode (e.g., images, PDFs).

FILE DETECTION:

OPEN: open is a functionality built into Python that allows you to open a file and utilize it in your program. The open function allows you to open a file such that you can read from it or write to it.

RELATIVE PATHS (test.txt) : This checks if test.txt exists in the same folder as the script.

ABSOLUTE PATHS (D:/test.txt): This checks if test.txt exists at D:/test.txt.

WITH: The `with` keyword ensures proper handling of files. It ****automatically closes the file**** when the block is exited.

```
name = input("What's your name? ")
# Open 'names.txt' in append mode ('a') and write the name
with open("names.txt", "a") as file:
    file.write(f'{name}\n') # Append name with a newline to separate entries
```

FILE WRITING/APPENDING:

WRITING:

Taking user input and writing to a file

```
name = input("What's your name? ")
```

Open the file in write mode ("w") - overwrites existing content

```
file = open("names.txt", "w")
file.write(name) # Write the name to the file
file.close() # Close the file
```

Note:

```
# The open function opens a file called "names.txt" with writing enabled, as signified by "w".
# This assigns the opened file to a variable called 'file'.
# The line file.write(name) writes the name to the text file.
# The line file.close() ensures the file is properly saved and closed.
# However, running this program multiple times will overwrite "names.txt" each time.
# To remove the existing text file, use the following command in the terminal:
# rm names.txt
```

APPENDING:

name = input("What's your name? ") # Get user input

```
file = open("names.txt", "a") # Open file in append mode ("a")
file.write(name) # Write the name to the file
file.close() # Close the file
```

TEXT FILE HANDLING:

```
txt_data = "I like pizza"
file_path = "output.txt"
```

Writing to a text file (overwrites if file already exists)

```
with open(file_path, "w") as file:
    file.write(txt_data) # Writing the data
print(f"TXT file '{file_path}' was created.")
```

Writing only if the file does NOT exist

```
try:
    with open(file_path, "x") as file:
        file.write(txt_data)
        print(f"TXT file '{file_path}' was created.")
except FileExistsError:
    print("File already exists.") # Avoids overwriting an existing file
```

Appending to a text file (Adds new content at the end)

```
try:
    with open(file_path, "a") as file:
        file.write("\n" + txt_data) # Adding new line before appending
        print(f"TXT file '{file_path}' was updated.")
except FileExistsError:
    print("File already exists.")
```

STORING LISTS IN TEXT FILES

```
employees = ["Eugene", "Squidward", "Patrick"]
file_path = "employees.txt"
```

```
# Writing list items to a text file, each on a new line
with open(file_path, "w") as file:
    for e in employees:
        file.write(e + "\n") # Writing each item on a new line

print(f"TXT file '{file_path}' was created.")
```

✦ JSON FILE HANDLING (DICTIONARY):

```
import json # Importing the JSON module

employees = {"name": "Eugene", "age": 30} # Dictionary to be stored
file_path = "output.json"

# Writing dictionary data as a JSON file
with open(file_path, "w") as file:
    json.dump(employees, file, indent=4) # Converts dictionary to JSON format

print(f"JSON file '{file_path}' was created.")
```

FILE READING:

✦ READING A TEXT FILE (.txt):

```
# Opening a file in read mode ('r') and reading all lines into a list
with open("names.txt", "r") as file:
    lines = file.readlines() # Reads all lines and stores them in a list
```

```
# Printing each name with an extra "hello," prefix
for line in lines:
    print("hello,", line) # ✗ This may create extra blank lines
```

```
# Fix: Using `rstrip()`
with open("names.txt", "r") as file:
    lines = file.readlines()
```

```
for line in lines:
    print("hello,", line.rstrip())
```

```
# Reading file line-by-line without storing in a list
with open("names.txt", "r") as file:
    for line in file:
        print("hello,", line.rstrip())
```

File Exceptions Handling

```
import os
file_path = "D:/test.txt" # File location (absolute path)
try:
    with open(file_path, "r") as file: # Open file in read mode ('r')
```

```

    content = file.read() # Read entire content of the file
    print(content) # Print file content
except FileNotFoundError:
    print("Error: File not found") # Handle missing file error
except PermissionError:
    print("Error: Permission denied") # Handle access issues

```

READING A JSON FILE (.json):

```

import json
file_path = "D:/test.json" # JSON file path
try:
    with open(file_path, "r") as file:
        content = json.load(file) # Load JSON data into a Python dictionary
        print(content) # Print full JSON data
        print(content["name"]) # Access a specific key-value pair
except FileNotFoundError:
    print("Error: File not found")
except PermissionError:
    print("Error: Permission denied")

```

SORTING FILE CONTENTS BEFORE PRINTING:

```

# Create an empty list to store names
names = []

```

```

# Read names from file and store in list after stripping newline characters
with open("names.txt", "r") as file:
    for line in file:
        names.append(line.rstrip()) # ☒ Removing newline characters

```

```

# Sorting names and printing in order
for name in sorted(names): # ☒ Alphabetically sorted names
    print(f"hello, {name}")

```

CSV (Comma Separated Values):

CSV FILE WRITING:

#Simple csv writing:

```

import csv # Importing the CSV module

employees = [
    ["Name", "Age", "Job"], # Column headers
    ["SpongeBob", 30, "Cook"],
    ["Patrick", 37, "Unemployed"],
    ["Sandy", 27, "Scientist"]
]

file_path = "output.csv"
# Writing data to a CSV file
with open(file_path, "w", newline="") as file:
    writer = csv.writer(file) # Creating a CSV writer object
    writer.writerows(employees) # Writing all rows at once

print(f"CSV file '{file_path}' was created.")

```

Writing To A Csv File Using Csv.Dictwriter

```
import csv

# Get user input
name = input("What's your name? ")
home = input("Where's your home? ")

# Open CSV file in append mode ('a') to add new entries
with open("students.csv", "a", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=["name", "home"]) # Define column headers
    writer.writerow({"name": name, "home": home}) # Write data as a dictionary

print("Data has been written to students.csv successfully!")
```

 READING A CSV FILE (.csv)

```
import csv

file_path = "D:/test.csv" # CSV file path

try:
    with open(file_path, "r") as file:
        content = csv.reader(file) # Read CSV content
        for line in content:
            print(line) # Print each row in CSV
except FileNotFoundError:
    print("Error: File not found")
except PermissionError:
    print("Error: Permission denied")
```

Reading A Csv File Using Csv.Reader

```
# Sample CSV file format:
# name,home
# Harry,"Number Four, Privet Drive"
# Ron,"The Burrow"
# Draco,"Malfoy Manor"

import csv

with open("students.csv") as file:
    reader = csv.reader(file) # Create a CSV reader object
    for row in reader: # Each row is a list
        print(f"{row[0]} lives in {row[1]}") # Access elements using index
```

Reading A Csv File Using Csv.Dictreader

```
students = [] # List to store student dictionaries

with open("students.csv") as file:
```

```

reader = csv.DictReader(file) # Reads each row as a dictionary
for row in reader: # Each row is a dictionary
    students.append({"name": row["name"], "home": row["home"]}) # Append dictionary to list

# Sorting students by name using a lambda function
for student in sorted(students, key=lambda student: student["name"]):
    print(f'{student["name"]} lives in {student["home"]}')

```

Reading A Csv File Line-By-Line

```

with open("students.csv") as file:
    for line in file:
        row = line.rstrip().split(",") # Remove trailing newline and split by comma
        print(f'{row[0]} is in {row[1]}') # Print student name and house

```

#Reading A Csv File With Named Variables

```

with open("students.csv") as file:
    for line in file:
        name, house = line.rstrip().split(",") # Unpack name and house from each row
        print(f'{name} is in {house}')

```

Sorting Csv Data As A Simple List

```

students = []

with open("students.csv") as file:
    for line in file:
        name, house = line.rstrip().split(",") # Read and split each line
        students.append(f'{name} is in {house}') # Append formatted string to list

for student in sorted(students): # Sort the list alphabetically
    print(student)

```

Storing Csv Data In A Dictionary

```

students = []

with open("students.csv") as file:
    for line in file:
        name, house = line.rstrip().split(",") # Read and split each row
        student = {} # Create an empty dictionary for each student
        student["name"] = name # Store name in dictionary
        student["house"] = house # Store house in dictionary
        students.append(student) # Append dictionary to the list

# Print each student's details
for student in students:
    print(f'{student["name"]} is in {student["house"]}')

```

Storing Csv Data In A List Of Dictionaries (Cleaner Version)

```

students = []

with open("students.csv") as file:
    for line in file:

```

```

name, house = line.rstrip().split(",")
students.append({"name": name, "house": house}) # Append dictionary directly

# Function to return a student's name for sorting
def get_name(student):
    return student["name"]

# Sort students by name and print
for student in sorted(students, key=get_name):
    print(f'{student["name"]} is in {student["house"]}')

```

LAMDA:

- **Lambda functions:** are anonymous (unnamed) functions used for simple, one-time operations.
- **Syntax:** lambda arguments: expression
 1. The `lambda` function replaces the need to define a separate `get\_name()` function.
 2. `lambda student: student["name"]` tells Python to use the "name" key for sorting.
 3. This makes the code **shorter** and **more readable**

#Lambda Function For Sorting Csv Data

```

# Sample CSV file format:
# name,house
# Harry,Gryffindor
# Ron,Gryffindor
# Draco,Slytherin

students = [] # List to store student dictionaries

# Reading CSV file and storing data in a list
with open("students.csv") as file:
    for line in file:
        name, house = line.rstrip().split(",") # Split each line into name & house
        students.append({"name": name, "house": house}) # Append dictionary to list

# Sorting students by name using a lambda function
for student in sorted(students, key=lambda student: student["name"]):
    print(f'{student["name"]} is in {student["house"]}')

```

BINARY FILE:

A binary file is a file that stores **raw binary data (0s and 1s)**.
This type of file can be used to store:

- Images (JPEG, PNG, GIF)
- Audio files (MP3, WAV)
- Video files (MP4, AVI)
- Any other non-text data.

Python has a library called **PIL (Pillow)** that works well with image files.

CREATING AN ANIMATED GIF USING PILLOW (PIL)

```

import sys # Allows working with command-line arguments
from PIL import Image # Importing Image module from PIL

# List to store loaded images

```

```

images = []

# Loop through command-line arguments (image file names)
for arg in sys.argv[1:]: # Skipping the script name (sys.argv[0])
    image = Image.open(arg) # Open each image file
    images.append(image) # Add image to list

# Save the first image as a GIF and append additional images to create animation
images[0].save(
    "costumes.gif", # Output GIF file name
    save_all=True, # Ensures all frames are saved
    append_images=[images[1]], # Appends second image to create animation
    duration=200, # Frame duration in milliseconds
    loop=0 # Loop indefinitely (0 = infinite loop)
)

```

HOW IT WORKS

1. `sys.argv` is used to get filenames from the command line.
2. `Image.open(arg)` opens each image and stores it in a list.
3. `images[0].save(...)` saves the first image and appends the second to it.
4. The `**duration**` specifies how long each frame is displayed.
5. `loop=0` makes the GIF loop `**infinitely**`.

HOW TO RUN THIS CODE

Open a terminal and run:
 python script.py costume1.gif costume2.gif
 This will create `costumes.gif`.

To view the GIF, run:

code costumes.gif (On VS Code or a compatible image viewer)

EXCEPTIONS HANDLING:

Exceptions are things that go wrong within our coding.

SYNTAX ERRORS:

Syntax errors are those that require you to double-check that you typed in your code correction.

RUNTIME ERRORS:

Runtime errors refer to those created by unexpected behavior within your code. For example, perhaps you intended for a user to input a number, but they input a character instead. Your program may throw an error because of this unexpected input from the user.

TRY AND EXCEPT:

In Python try and except are ways of testing out user input before something goes wrong.

EXAMPLES:

def main():

```

    x = get_int("Enter a number: ")
    print(f"x is {x}")

```

Function to get a valid integer from the user

```
def get_int(prompt):
    while True:
        try:
            return int(input(prompt)) # Return the input if it's a valid integer
        except ValueError:
            print("x is not an integer") # Display an error and re-ask
```

Example of basic try-except handling

```
try:
    x = int(input("What's x? "))
    print(f"x is {x}")
except ValueError:
    print("x is not an integer")
```

Try-except with an else block

```
try:
    x = int(input("What's x? "))
except ValueError:
    print("x is not an integer")
else:
    print(f"x is {x}")
```

Keep asking until a valid integer is provided

```
while True:
    try:
        x = int(input("What's x? "))
    except ValueError:
        print("x is not an integer")
    else:
        break # Exit loop if input is valid
```

```
print(f"x is {x}")
```

Function-based approach with loop validation

```
def get_int_with_message():
    while True:
        try:
            x = int(input("What's x? "))
            return x # Return valid integer and exit loop
        except ValueError:
            print("x is not an integer")
```

```
x = get_int_with_message()
print(f"x is {x}")
```

Function with silent error handling (just re-asks)

```
def get_int_silent(prompt):
    while True:
        try:
            return int(input(prompt)) # Return input if valid
        except ValueError:
```

```

pass # No message, just re-asks

x = get_int_silent("What's x? ")
print(f"x is {x}")

# Run the main function to demonstrate using a custom prompt
main()

#using try,except,finally

try:
    number = int(input("Enter a number: ")) # Convert input to an integer
    print(1 / number) # Attempt division
except ZeroDivisionError:
    print("You can't divide by zero, idiot!") # Handles division by zero
except ValueError:
    print("Enter only numbers, please.") # Handles non-numeric input
except Exception:
    print("Something went wrong.") # Catches any other unexpected errors
finally:
    print("Doing some cleanup...") # Always executes (useful for file handling e.g automatically closing file)

```

DEBUGGING AND TROUBLESHOOTING:

WHAT IS TROUBLESHOOTING?

Troubleshooting is the process of identifying, analyzing, and solving problems. We can use the term troubleshooting to refer to solving any kind of problem.

Troubleshooting when we're fixing problems in the system running the application, and debugging when we're fixing the bugs in the actual code of the application. There are lots of tools that we can use to get more information about the system and what the programs in our system are doing.

Tools like *tcpdump* and *wireshark* can show us ongoing network connections, and help us analyze the traffic going over our cables. Tools like *ps*, *top*, or *free* can show us the number and types of resources used in the system. We can use a tool like *strace* to look at the system calls made by a program, or *ltrace* to look at the library calls made by the software.

WHAT IS DEBUGGING?

Debugging is the process of identifying, analyzing, and removing bugs in a system. When debugging the code of a program, we can combine these tools with specific debugging tools developed for the programming language used to write the application. Debuggers let us follow the code line by line, inspect changes in variable assignments, interrupt the program when a specific condition is met, and more

PROBLEM SOLVING STEPS:

The first step is **getting information**. This means gathering as much information as we need about the current state of things, what the issue is, when it happens, and what the consequences are, for example. To get this information, we can use any existing documentation that might help. This can be internal documentation, manual pages, or even questions asked on the Internet. One super important resource to solve a problem is the reproduction case, which is a clear description of how and when the problem appears.

The second step is **finding the root cause of the problem**. This is usually the most difficult step.

The final step is **performing the necessary remediation**. Depending on the problem, this might include an immediate remediation to get the system back to health, and then a medium or long-term remediation to avoid the problem in the future.

LIBRARIES:

Libraries are bits of code written by you or others can you can use in your program.

Python allows you to share functions or features with others as “modules”.

HOW DO YOU LOAD A MODULE INTO YOUR OWN PROGRAM?

You can use the word ‘import’ in your program

```
import library
```

‘from’ allows us to be very specific about what we’d like to import.

```
from library import module
```

RANDOM MODULE:

```
import random
```

```
# Randomly choose between "heads" and "tails"
```

```
coin = random.choice(["heads", "tails"])
print(coin)
```

```
# Using 'choice' from random module
```

```
from random import choice
coin = choice(["heads", "tails"])
print(coin)
```

```
# Generate a random integer between a and b
```

```
number = random.randint(1, 10)
print(number)
```

```
# Shuffle a list into a random order
```

```
cards = ["jack", "queen", "king"]
random.shuffle(cards)
```

```
# Print shuffled cards
```

```
for card in cards:
```

```

    print(card)
#random floating point
    num=random.random()

```

STATISTICS MODULE:

Python comes with a built-in statistics library. mean is a function of this library that is quite useful.

```

import                statistics

print(statistics.mean([100, 90]))

```

SYS MODULE:

So far, we have been providing all values within the program that we have created. What if we wanted to be able to take input from the command-line? For example, rather than typing python average.py in the terminal, what if we wanted to be able to type python average.py 100 90 and be able to get the average between 100 and 90? **sys is a module** that allows us to take arguments at the command line. **argv is a function within the sys module** that allows us to learn about what the user typed in at the command line. Notice how you will see sys.argv utilized in the code below.

```

import sys

print("hello, my name is", sys.argv[1]) What do you think is held currently in sys.argv[0]? If
you guessed name.py, you would be correct!
import sys

```

Argument validation

```

if len(sys.argv) < 2:
    print("Too few arguments")
elif len(sys.argv) > 2:
    print("Too many arguments")
else:
    print("hello, my name is", sys.argv[1])

```

Using sys.exit for cleaner exits

```

import sys
if len(sys.argv) < 2:
    sys.exit("Too few arguments")
elif len(sys.argv) > 2:
    sys.exit("Too many arguments")
print("hello, my name is", sys.argv[1])

```

Handling multiple arguments

```

import sys
if len(sys.argv) < 2:
    sys.exit("Too few arguments")
for arg in sys.argv[1:]:
    print("hello, my name is", arg)

```

APIs (REQUESTS MODULE):

APPLE ITUNES:

```
import json    # Importing JSON library for formatting the API response
import requests # Importing requests to make API calls
import sys     # Importing sys to handle command-line arguments
```

Checking For Correct Usage (Must Provide A Search Term)

```
if len(sys.argv) != 2:
    print("Usage: python itunes.py <search-term>")
    sys.exit() # Exit if incorrect arguments are provided
```

Making A Request To Apple's Itunes Api

```
search_term = sys.argv[1] # Getting the search term from command-line input
url = f"https://itunes.apple.com/search?entity=song&limit=50&term={search_term}"
response = requests.get(url) # Sending request to API
```

Displaying The Raw Json Response (Formatted)

```
data = response.json() # Convert response to Python dictionary
print(json.dumps(data, indent=2)) # Pretty-print JSON response for readability
```

Extracting & Printing Song Names From Api Response

```
print("\n 🎵 List of Songs Found:")
for result in data["results"]:
    print(f" ✓ {result['trackName']}") # Extract and print only the song name
```

POKEMON API:

- response is object that returns status code
- Informational responses (100 – 199)
- Successful responses (200 – 299)
- Redirection messages (300 – 399)
- Client error responses (400 – 499)
- Server error responses (500 – 599)

This script fetches Pokémon details from the PokéAPI.

```
import requests
# Base Api Url
BASE_URL = "https://pokeapi.co/api/v2/"
# Function To Fetch Pokémon Data
def get_pokemon_info(name):
    """Fetch Pokémon information from PokéAPI using the given Pokémon name."""
    url = f"{BASE_URL}pokemon/{name.lower()}" # Construct API URL (convert name to lowercase)
    response = requests.get(url) # Make the API request

    # Check if the request was successful
    if response.status_code == 200:
        return response.json() # Convert JSON response to Python dictionary
    else:
        print(f" ⚠ Failed to retrieve data. Status Code: {response.status_code}")
        return None
```

Fetching Data For A Specific Pokémon

```
pokemon_name = "pikachu" # Change this to any Pokémon name
pokemon_info = get_pokemon_info(pokemon_name)
```

Displaying Pokémon Data

```

if pokemon_info:
    print("\n 🐾 Pokémon Details:")
    print(f" ✓ Name: {pokemon_info['name'].capitalize()}")
    print(f" ✓ ID: {pokemon_info['id']}")
    print(f" ✓ Height: {pokemon_info['height']}")
    print(f" ✓ Weight: {pokemon_info['weight']}")

```

DATE-TIME MODULE:

```
import datetime
```

Creating A Specific Date

```

date = datetime.date(2025, 1, 2) # Year, Month, Day
print("Specific Date:", date)

```

Getting Today's Date

```

today = datetime.date.today() # Gets the current date
print("Today's Date:", today)

```

Creating A Specific Time

```

time = datetime.time(12, 30, 0) # Hours, Minutes, Seconds
print("Specific Time:", time)

```

Getting Current Date & Time (Timestamp)

```

now = datetime.datetime.now() # Gets current date & time
formatted_now = now.strftime("%H:%M:%S %m-%d-%Y") # Formatting (HH:MM:SS MM-DD-YYYY)
print("Current Date & Time (Formatted):", formatted_now)

```

Comparing Dates & Times

```

target_datetime = datetime.datetime(2030, 1, 2, 12, 30, 1) # Future date & time
current_datetime = datetime.datetime.now() # Current date & time

```

```
# Check if target date has passed
```

```

if target_datetime < current_datetime:
    print("Target Date has PASSED.")
else:
    print("Target Date has NOT PASSED.")

```

MULTI-THREADING MODULE:

- **Threading allows functions to execute simultaneously** instead of one after another.
- **threading.Thread(target=function_name)** → Creates a thread.
- **.start()** → Begins execution of the function in a separate thread.
- **.join()** → Ensures the main program waits for the thread to finish.
- **Tasks execute in parallel** → The one that finishes first prints first.
- **Passing arguments** → Use args=("value",) (note the comma for a single argument tuple).

```

import threading # Importing the threading module
import time      # Importing time for simulating task delays

```

Defining Multiple Functions (Tasks)

```

def walk_dog():
    """Simulate walking the dog (takes 8 seconds)."""
    time.sleep(8)

```

```

print("✔ You finished walking the dog")

def take_out_trash():
    """Simulate taking out the trash (takes 2 seconds)."""
    time.sleep(2)
    print("✔ You took out the trash")

def get_mail():
    """Simulate getting the mail (takes 4 seconds)."""
    time.sleep(4)
    print("✔ You got the mail")

# Creating Threads For Multi-Tasking
chore1 = threading.Thread(target=walk_dog)
chore2 = threading.Thread(target=take_out_trash)
chore3 = threading.Thread(target=get_mail)

# Starting Threads (Executing Functions Simultaneously)
chore1.start()
chore2.start()
chore3.start()

# Waiting For All Threads To Finish
chore1.join() # Ensures this thread finishes before continuing
chore2.join()
chore3.join()

print("🎉 All chores are complete!") # Runs only after all tasks finish

# Passing Arguments To A Thread Function (Example)
# chore2 = threading.Thread(target=take_out_trash, args=("scooby",))
# args must be a tuple (even with one element, add a comma)

```

HOW TO INSTALL LIBRARY?

PyPI is a repository or directory of all available third-party packages currently available. Python has a package manager called pip that allows you to install packages quickly onto your system. you can install the cowsay package by typing **pip install cowsay**.

HOW TO CREATE YOUR OWN LIBRARY:

```

sayings.py:
def hello(name):
    print(f"hello, {name}")

def goodbye(name):
    print(f"goodbye, {name}")

say.py:

```

```
import sys
from sayings import goodbye

# Ensure a name is provided as a command-line argument
if len(sys.argv) == 2:
    goodbye(sys.argv[1])
```

UNIT TEST:

calculator.py:

```
def main():
    x = int(input("What's x? "))
    print("x squared is", square(x))

def square(n):
    return n * n

if __name__ == "__main__":
    main()
```

test_calculator.py:

```
from calculator import square

def main():
    test_square()

def test_square():
    if square(2) != 4:
        print("2 squared was not 4")
    if square(3) != 9:
        print("3 squared was not 9")

if __name__ == "__main__":
    main()
```

ASSERT COMMAND:

The assert statement checks whether a condition is True. If it's False, Python raises an AssertionError.

assert condition, "Error message (optional)"

```
from calculator import square

def main():
    test_square()

def test_square():
    assert square(2) == 4, "Error: 2 squared is not 4"
    assert square(3) == 9, "Error: 3 squared is not 9"
    assert square(-3) == 9, "Error: -3 squared is not 9"
    assert square(0) == 0, "Error: 0 squared is not 0"

if __name__ == "__main__":
    main()
```

PYTEST:

- pytest is a third-party Python testing framework.
- It makes **writing, running, and debugging tests** easier.
- It provides **detailed error messages** and supports **fixtures, parameterized tests, and test discovery**.

1. calculator.py

```
def square(n):
    return n * n
```

2. test_calculator.py

```
from calculator import square
```

```
def test_square():
    assert square(2) == 4
    assert square(3) == 9
    assert square(-2) == 4
    assert square(-3) == 9
    assert square(0) == 0
```

- Each assert statement **checks** if square(n) returns the correct result.
- If any assertion **fails**, pytest will display a **detailed error message**

3. Running Pytest:

To run the test file, type in the term
 pytest test_calculator.py

4. Expected Output (if all tests pass):

```
===== test session starts =====
collected 1 item

test_calculator.py .                [100%]

===== 1 passed in 0.12s =====
```

5. If a test fails, pytest provides useful output:

- ```
> assert square(-3) == 8
 • This shows the exact line where the test failed.
 • The test expected square(-3) to return 8, but something else happened.
```
- ```
E      assert 9 == 8
    • E stands for Error.
    • It shows that the actual result (9) did not match the expected result (8).
```
- ```
E + where 9 = square(-3)
 • This tells us that square(-3) returned 9, not 8.
 • This means there's likely a mistake in our expected output rather than the function itself.
```

This tells us exactly which **assertion failed** and what was expected vs. what was returned.

**ORGANIZING TEST FOR MORE DETAILED TEST:**

Dividing test cases into groups makes debugging easier and helps isolate issues. Below is the modified test\_calculator.py:

```
from calculator import square
```

```
Test cases for positive numbers
```

```
def test_positive():
 assert square(2) == 4
 assert square(3) == 9
```

```
Test cases for negative numbers
```

```
def test_negative():
 assert square(-2) == 4
 assert square(-3) == 9
```

```
Test case for zero
```

```
def test_zero():
 assert square(0) == 0
```

- ☒ If all tests pass, you'll see output like:
- 3 passed in 0.01s
- ☒ If a test fails, Pytest will highlight which test function failed.

### **RAISE COMMAND:**

Using **pytest.raises** to Test Errors. The raises function in pytest is used to check whether a specific error is raised when executing a function. This is particularly useful when testing for invalid inputs. raises which allows you to express that you expect an error to be raised. We need to go to the top of our program and add import pytest and then call pytest.raises with the type of error we are expecting.

```
import pytest
from calculator import square
```

```
Test cases for positive numbers
```

```
def test_positive():
 assert square(2) == 4
 assert square(3) == 9
```

```
Test cases for negative numbers
```

```
def test_negative():
 assert square(-2) == 4
 assert square(-3) == 9
```

```
Test case for zero
```

```
def test_zero():
 assert square(0) == 0
```

```
Test case for invalid string input
```

```
def test_str():
 with pytest.raises(TypeError):
 square("cat")
```

### **TESTING STRINGS:**

- print() displays output but does not return a value.
- return sends a value back to the caller, making testing easier.

- Functions that return values can be **directly asserted** in test cases.
- Easier to compare expected and actual results programmatically.

### #Problem with the Initial Approach

```
def hello(to="world"):
 print("hello,", to)
assert hello("David") == "hello, David"
will fail because hello("David") returns None (since print() doesn't return a value).
```

### #Fixing the hello Function

```
def hello(to="world"):
 return f"hello, {to}"
from hello import hello
def test_hello():
 assert hello("David") == "hello, David"
 assert hello("Alice") == "hello, Alice"
 assert hello() == "hello, world"
```

## ORGANIZING TESTS INTO FOLDERS:

### Steps to Organize Tests into Folders in Python:

- Create a **test** folder: Run: `mkdir test`
- Create a test file inside the folder: Run: `code test/test_hello.py`
- Write test cases inside **test/test\_hello.py**
- Create an `__init__.py` file in the test folder: Run: `code test/__init__.py`. Leave it empty (this makes the folder recognizable for pytest).
- Run all tests inside the test folder: Run: `pytest test`

## REGULAR EXPRESSION:

Regular expressions (regex) help identify patterns in text, such as validating email formats, phone numbers, or specific word structures. Python provides the `re` module to work with regex efficiently.

- Used for pattern matching in strings
- Helps validate user inputs (e.g., checking if an email is formatted correctly)
- Python provides the **re** module with functions like `search()`, `match()`, `findall()`, and `sub()`

### RE.SEARCH LIBRARY:

The `re.search()` function is one of the most versatile tools in the `re` module. It scans a string for the first occurrence of a pattern and returns a **match object** if found; otherwise, it returns `None`.

**`re.search(pattern, string, flags=0)`.**

## COMMON REGULAR EXPRESSION SYMBOLS & PATTERNS

Regular expressions (regex) provide a powerful way to search, match, and manipulate text. Below is a list of **commonly used regex symbols** along with their meanings.

### Basic Matching:

- `.` → Matches **any character** except a newline (`\n`).
- `^` → Matches the **start** of a string.
- `$` → Matches the **end** of a string (or just before a newline).

### Repetitions:

- `*` → **0 or more** repetitions (e.g., `a*` matches `a`, `aa`, `aaa`, etc.).

- **+** → **1 or more** repetitions (e.g., `a+` matches `a`, `aa`, `aaa`, but not an empty string).
- **?** → **0 or 1** repetition (e.g., `colou?r` matches both `"color"` and `"colour"`).
- **{m}** → Exactly **m repetitions** (e.g., `a{3}` matches `"aaa"`).
- **{m,n}** → Between **m and n repetitions** (e.g., `a{2,4}` matches `"aa"`, `"aaa"`, or `"aaaa"`).

### Character Classes:

- **[abc]** → Matches **any one of the characters** `a`, `b`, or `c`.
- **[^abc]** → Matches **any character except** `a`, `b`, or `c`.
- **\d** → Matches **any digit** (0-9).
- **\D** → Matches **any non-digit** character.
- **\s** → Matches **whitespace characters** (space, tab, newline).
- **\S** → Matches **non-whitespace characters**.
- **\w** → Matches **word characters** (`a-z`, `A-Z`, `0-9`, `_`).
- **\W** → Matches **non-word characters**.

### Groups & Alternation:

- **A|B** → Matches **either A or B**.
- **(abc)** → **Capturing group** (treats `abc` as a unit).
- **(?:abc)** → **Non-capturing group** (same as `(abc)`, but does not store the match).

### EMAIL VALIDATION:

```
import re
```

```
Prompt the user for an email address and remove any leading/trailing spaces
```

```
email = input("What's your email? ").strip()
```

```
Use regex to check if the email contains at least one character before and after '@'
```

```
if re.search(".*@.*", email):
```

```
 print("Valid")
```

```
else:
```

```
 print("Invalid")
```

```
Explanation of the regex pattern:
```

```
".*@.*"
```

```
- ".*" ensures there is at least one character before '@'
```

```
- "@" ensures the presence of the '@' symbol
```

```
- ".*" ensures there is at least one character after '@'
```

```
Running this code:
```

```
- If input is "malan@", the output will be "Invalid" as expected
```

```
- If input is "user@example.com", the output will be "Valid"
```

```
Note: This regex only performs a basic check and does not validate full email format.
```

### ESCAPE CHARACTER and RAW STRINGS:

#### • Escape Character (\)

- `.` normally matches any character in regex.
- `\.` ensures that we are looking for a **literal** dot (`.`), not a wildcard character.

#### • Raw Strings (r" ")

- Prevents Python from interpreting `\` as an escape sequence.
- Ensures `\.` is treated as `\.` and not interpreted differently.

- Similar to `f""` format strings, `r""` ensures special characters are taken **literally**.

```
import re
```

```
Prompt user for email input and remove extra spaces
```

```
email = input("What's your email? ").strip()
```

```
Using regex to check if the email follows the pattern ".+@.+\.edu"
```

```
Explanation:
```

```
- `.+` → Ensures there is at least one character before and after "@"
```

```
- `@` → Ensures the presence of "@" symbol
```

```
- `\. ` → The backslash ` ` escapes the dot, ensuring it is treated as a literal dot
```

```
- `edu` → Ensures the domain ends with "edu"
```

```
- `r""` → A raw string is used so that `\. ` is read literally
```

```
if re.search(r".+@.+\.edu", email):
```

```
 print("Valid") # If the pattern matches, email is valid
```

```
else:
```

```
 print("Invalid")#Otherwise,it'sinvalid
```

## **START AND END ANCHORS ^ \$ :**

**Start (^) and End (\$) Anchors:**

- `^` → Forces the pattern to begin at the start of the string.
- `$` → Forces the pattern to end at the last character of the string.
- Ensures no extra text before or after the valid email.

**Difference from Previous Regex (`r".+@.+\.edu"`)**

- Previous version matched valid emails anywhere in a string.
- This version ensures the entire input is a valid email.

```
import re
```

```
Prompt user for email input and remove extra spaces
```

```
email = input("What's your email? ").strip()
```

```
Using regex to validate email with start (^) and end ($) anchors
```

```
Explanation:
```

```
- `^` → Ensures the match starts at the beginning of the string
```

```
- `.+` → Ensures at least one character before and after "@"
```

```
- `@` → Ensures presence of "@" symbol
```

```
- `\. ` → Escapes the dot to ensure it's treated as a literal dot
```

```
- `edu` → Ensures domain ends with "edu"
```

```
- `$` → Ensures the match ends at the last character of the string
```

```
if re.search(r"^.+@.+\.edu$", email):
```

```
 print("Valid") # If the pattern matches the entire string, it's valid
```

```
else:
```

```
 print("Invalid") # If there are extra characters before or after, it's invalid
```

## **COMPLIMENTING:**

- Complimenting means that all characters except that one character
- `[^@]` → Complement Set, meaning any character except "@"

- `[^@]+` → One or more characters except "@" before and after @
- Ensures there's exactly one "@" in the email address.

```
import re
```

```
Prompt user for email input and remove extra spaces
```

```
email = input("What's your email? ").strip()
```

```
Using regex to validate email with complementing character set
```

```
Explanation:
```

- # - `^` → Ensures the match starts at the beginning of the string
- # - `^[^@]+` → Ensures at least one character **except** "@" before the "@" symbol
- # - `@` → Ensures the presence of exactly one "@"
- # - `[^@]+` → Ensures at least one character **except** "@" after "@"
- # - `\.` → Escapes the dot to ensure it's treated as a literal dot
- # - `edu` → Ensures the domain ends with "edu"
- # - `$` → Ensures the match ends at the last character of the string

```
if re.search(r"^[^@]+@[^@]+\.\edu$", email):
```

```
 print("Valid") # If the pattern matches the entire string, it's valid
```

```
else:
```

```
 print("Invalid") # If multiple "@" exist or other issues arise, it's invalid
```

### **NOT COMPLIMENTING:**

**Not complementing** means we specify **what characters are allowed** instead of excluding certain ones.

This ensures that input follows a strict format by allowing only specific characters.

**Using `[a-zA-Z0-9_]` :**

- `[a-zA-Z0-9_]` → Only allows letters, numbers, and underscores before @ and after it.
- `@` → Ensures exactly one "@"
- `\.edu` → Ensures the domain ends with ".edu"

```
import re
```

```
email = input("What's your email? ").strip()
```

```
Allows only letters (a-z, A-Z), numbers (0-9), and underscores (_)
```

```
if re.search(r"^[a-zA-Z0-9_]+@[a-zA-Z0-9_]+\.\edu$", email):
```

```
 print("Valid")
```

```
else:
```

```
 print("Invalid")
```

**Using `\w` (Shorthand for `[a-zA-Z0-9_]`):**

- `\w+` → Matches one or more word characters (letters, numbers, and underscores).
- `@` → Ensures exactly one "@"
- `\w+` → Ensures a valid domain name
- `\.edu` → Ensures the email ends in ".edu"

```
import re
```

```
email = input("What's your email? ").strip()
```

```
\w automatically includes a-z, A-Z, 0-9, and _
```

```
if re.search(r"^\w+@\w+\.\edu$", email):
```

```

 print("Valid")
else:
 print("Invalid")

```

### **OR EXPRESSION:**

The **| (OR operator)** allows us to match multiple possible values. This is useful when we want to validate multiple domain types (e.g., .com, .edu, .gov, etc.).

```

import re # Import the regular expressions library

Prompt user for an email address and remove leading/trailing spaces
email = input("What's your email? ").strip()

Regular expression to match valid email formats with specific domain extensions
if re.search(r"^\w+@\w+\.(com|edu|gov|net|org)$", email):
 print("Valid") # If the pattern matches, print "Valid"
else:
 print("Invalid") # If the pattern doesn't match, print "Invalid"

```

#### **Explanation of the Regular Expression (r"^\w+@\w+\.(com|edu|gov|net|org)\$")**

- **^** → Ensures the match starts from the beginning of the string.
- **\w+** → Matches one or more word characters (**letters, digits, underscore**). This ensures a valid username before **@**.
- **@** → Matches the literal **@** symbol in the email address.
- **\w+** → Matches one or more word characters (**letters, digits, underscore**) for the domain name.
- **\.** → Matches a **literal dot (.)** before the domain extension.
- **(com|edu|gov|net|org)** → Matches **one of** the specified domain extensions (.com, .edu, .gov, .net, or .org).
- **\$** → Ensures the match **ends** at the last character of the string (no extra characters allowed).

### **FLAGS:**

- **re.IGNORECASE** : Makes the match **case-insensitive** (e.g., HARVARD.EDU = harvard.edu)
- **re.MULTILINE** : Allows **^** (start) and **\$** (end) to match at each **line** in multi-line input
- **re.DOTALL** : Allows **.** to match **newlines (\n)**, not just regular characters

```

import re # Import the regular expressions library
Prompt user for an email address and remove leading/trailing spaces
email = input("What's your email? ").strip()

```

```

Regular expression to match valid email formats (only .edu domains)
The re.IGNORECASE flag makes the search case-insensitive

```

```

if re.search(r"^\w+@\w+\.edu$", email, re.IGNORECASE):
 print("Valid") # If the pattern matches, print "Valid"
else:
 print("Invalid") # If the pattern doesn't match, print "Invalid"

```

#### **Explanation of the Code & Regular Expression (r"^\w+@\w+\.edu\$"):**

- **^** → Ensures the match starts from the beginning of the string.
- **\w+** → Matches one or more **word characters** (letters, digits, underscore) for the username.
- **@** → Matches the **literal @** symbol in the email address.
- **\w+** → Matches one or more word characters for the domain name.

- `\.` → Matches a **literal dot (.)** before the domain extension.
- `edu` → Ensures the email ends with `.edu` (academic domain).
- `$` → Ensures the match **ends** at the last character (no extra characters allowed).
- `re.IGNORECASE` → Makes the regex **case-insensitive**, allowing emails like `MALAN@HARVARD.EDU` to be valid.

### ? EXPRESSION:

- Adding `(\w+\.)?` makes the subdomain optional

```
import re # Import the regular expressions library
```

```
Prompt user for an email address and remove leading/trailing spaces
email = input("What's your email? ").strip()
```

```
Regular expression to match valid email formats (allowing optional subdomains)
The re.IGNORECASE flag makes the search case-insensitive
```

```
if re.search(r"^\w+@(\w+\.)?\w+\.edu$", email, re.IGNORECASE):
 print("Valid") # If the pattern matches, print "Valid"
else:
 print("Invalid") # If the pattern doesn't match, print "Invalid"
```

### **Explanation of the Regular Expression `(r"^\w+@(\w+\.)?\w+\.edu$")`:**

- `^` → Ensures the match starts from the beginning of the string.
- `\w+` → Matches one or more **word characters** (letters, digits, underscore) for the username.
- `@` → Matches the **literal @** symbol in the email address.
- `(\w+\.)?` → Matches **zero or one occurrence** (? operator) of a subdomain (`cs50.` in `malan@cs50.harvard.edu`).
- `\w+` → Matches one or more **word characters** for the subdomain name (e.g., `cs50`).
- `\.` → Matches a **literal dot (.)** after the subdomain (e.g., `cs50.`).
- `?` → Makes the entire **subdomain group optional**, meaning it can appear **once or not at all**.
- `\w+` → Matches the **main domain name** (e.g., `harvard`).
- `\.` → Matches the **literal dot (.)** before the domain extension.
- `edu` → Ensures the email ends with `.edu`.
- `$` → Ensures the match **ends** at the last character (no extra characters allowed).
- `re.IGNORECASE` → Makes the regex **case-insensitive**, allowing emails like `MALAN@CS50.HARVARD.EDU` to be valid.

### EMAIL VALIDATION EXPRESSION:

here is the full expression that one would have to type to ensure that a valid email is inputted

```
^[a-zA-Z0-9.!#$%&'*+V=?^_`{|}~-]+@[a-zA-Z0-9](?:[a-zA-Z0-9]{0,61}[a-zA-Z0-9])?(?:\.[a-zA-Z0-9](?:[a-zA-Z0-9]{0,61}[a-zA-Z0-9])?)$
```

- `re.match(pattern, string)` → Checks if the string **starts** with a valid email.
- `re.fullmatch(pattern, string)` → Ensures the **entire string** is a valid email.

### CLEANING UP USER INPUT:

When accepting user input, **we should never assume that users will format data correctly.**

Here, we ensure that names are **properly formatted** by detecting if they are entered in "Last, First" format and reordering them.

**Approach 1: Using `split()` :**

```
Basic approach using split()
name = input("What's your name? ").strip()

Check if the name contains a comma
if "," in name:
 last, first = name.split(", ") # Split into last and first name
 name = f"{first} {last}" # Reorder to "First Last"

print(f"hello, {name}")
```

**Issues with this approach:**

- Assumes users will always type ", " (comma with space)
- **Fails** if there are multiple spaces or no space after the comma

**Approach 2: Using `re.search()` :**

```
import re
Get user input
name = input("What's your name? ").strip()

Regex pattern to capture "Last, First"
matches = re.search(r"^(.+), (.+)$", name)

If the pattern matches, reorder the name
if matches:
 last, first = matches.groups() # Extract matched groups
 name = first + " " + last # Convert to "First Last"

print(f"hello, {name}")
```

**Explanation of `re.search(r"^(.+), (.+)$", name)`:**

Pattern Meaning

- ^      Start of the string
- (.+)   Captures any characters (First Name or Last Name)
- ,
- (.+)   Captures the second part (First Name or Last Name)
- \$      End of the string

✓ "Smith, Alice" → "Alice Smith"

✗ "Doe,John" (Fails because there's no space after the comma)

**Approach 3: Handling Extra Spaces with \* in Regex:**

**\* after ,** → Allows **zero or more spaces**

```
import re

Get user input
name = input("What's your name? ").strip()

Regex pattern to handle spaces around the comma
matches = re.search(r"^(.+), *(.+)$", name) # `*` allows multiple spaces after `,`
```

```
If the pattern matches, reorder the name
if matches:
 name = matches.group(2) + " " + matches.group(1) # Convert to "First Last"

print(f"hello, {name}")
```

### **WALRUS:**

Python 3.8+ introduced the **walrus operator** (`:=`), which allows you to **assign a value and check a condition in the same expression**.

```
import re
```

```
Get user input
name = input("What's your name? ").strip()
```

```
Use the walrus operator to combine assignment and condition checking
if matches := re.search(r"^(.+), *(.+)$", name):
 name = matches.group(2) + " " + matches.group(1) # Convert "Last, First" to "First Last"

print(f"hello,{name}")
```

### **How It Works:**

|                                    |                                                                                                   |
|------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>matches := re.search(...):</b>  | matches is assigned the result of re.search(...) <b>while</b> being evaluated in the if condition |
| <b>if matches:</b>                 | If matches is <b>not None</b> , the condition is True                                             |
| <b>.group(2) + " " + .group(1)</b> | Swaps "Last, First" to "First Last"                                                               |

### **EXTRACTING USER IN PUT**

```
Get URL input from the user
```

```
url = input("URL: ").strip()
Print the full URL
print(url)
```

**#Instead of displaying the full URL, let's extract only the username from the Twitter URL:**

```
Get URL input from user
url = input("URL: ").strip()

Remove "https://twitter.com/" from the URL
username = url.replace("https://twitter.com/", "")

Print the extracted username
print(f"Username: {username}")
```

### **REMOVE PREFIX:**

- It **only removes** the specified prefix if it **appears at the start** of the string.
- If the string **doesn't start** with the prefix, it remains unchanged.

```
url = input("URL: ").strip()

Remove the fixed prefix "https://twitter.com/"
username = url.removeprefix("https://twitter.com/")

print(f"Username: {username}")
```

### **SUB LIBRARY:**

- The `re.sub()` method in Python's `re` library is used to **replace** a pattern with another string
- **`re.sub(pattern, replacement, string, count=0, flags=0)`**
- **pattern** → The regex pattern to search for.
- **replacement** → The string to replace the matched pattern.
- **string** → The input string to search and replace.
- **count (optional)** → Maximum number of replacements.
- **flags (optional)** → Additional flags like `re.IGNORECASE`.

#### **Example 1: Basic `re.sub()` :**

```
import re

url = input("URL: ").strip()

Remove "https://twitter.com/" from the input
username = re.sub(r"https://twitter.com/", "", url)
print(f"Username: {username}")
```

#### **Example 2: Handling `http`, `https`, and `www`:**

```
import re

url = input("URL: ").strip()

Remove "http", "https", "www", and "twitter.com/"
username = re.sub(r"^(https?:/)?(www\.)?twitter\.com/", "", url)

print(f"Username: {username}")
```

#### **Example 3: Using `re.search()` :**

```
import re

url = input("URL: ").strip()

Extract username using regex groups
matches = re.search(r"^(https?:/)?(www\.)?twitter\.com/(.+)$", url, re.IGNORECASE)

if matches:
 print(f"Username:", matches.group(2)) # Group 2 contains the username
```

- `(.+) $` → Captures the **username** after `twitter.com/`
- `re.IGNORECASE` → Makes the pattern **case-insensitive** (Twitter.com, TWITTER.COM all work)



### **Conclusion:**

- Use **`re.sub()`** if you just want to remove prefixes.
- Use **`re.search()`** if you need to extract specific parts of the string.
- Use regex flags like **`re.IGNORECASE`** to make matching more flexible.

**?: EXPRESSION:**

- In regular expressions, (?:...) is used to create a **non-capturing group**. By default, **parentheses () create a capturing group**, meaning the matched content can be extracted using .group(n). However, sometimes we don't need to **store** a part of the match—we just want to use it for **matching**.
- **Capturing Group:** () → Stores the matched content
- **Non-Capturing Group:** (?:...) → Matches the pattern but does **not** store it

**#Extracting Twitter Username Using Regex:**

```
import re # Import the regular expressions module

Get URL input from the user and remove any leading/trailing spaces
url = input("URL: ").strip()

Regular expression to match a valid Twitter profile URL
if matches := re.search(r"^https?://(?:www\.)?twitter\.com/([a-z0-9_]+)$", url, re.IGNORECASE):
 # If a match is found, extract and print the username
 print(f"Username:", matches.group(1))
```

**FINAL PROJECT:**

This script processes a list of **login/logout events** and **generates a report** showing **which users are currently logged into which machines**

**# Define a helper function to extract the date from an event**

```
def get_event_date(event):
 return event.date
```

```
def current_users(events):
```

```
 """
```

```
 Processes login and logout events to determine which users are
 currently logged in on each machine.
```

```
 """
```

**# Sort events by date to ensure correct processing order**

```
events.sort(key=get_event_date)
```

**# Dictionary to store logged-in users on each machine**

```
machines = {}
```

**# Process each event**

```
for event in events:
```

```
 # If the machine is not already in the dictionary, initialize it
```

```
 if event.machine not in machines:
```

```
 machines[event.machine] = set() # Use a set to avoid duplicate users
```

**# Add or remove users based on login/logout events**

```
 if event.type == "login":
```

```
 machines[event.machine].add(event.user)
```

```
 elif event.type == "logout":
```

```
 machines[event.machine].discard(event.user) # Use discard to avoid errors if user not in set
```

```
return machines
```

```
def generate_report(machines):
```

```

"""
Prints a report of all machines and their currently logged-in users.
"""
for machine, users in machines.items():
 if users:
 user_list = ", ".join(users)
 print(f"{machine}: {user_list}")
 else:
 print(f"{machine}: No users currently logged in")

```

### # Define the Event class

```

class Event:
 def __init__(self, date, machine, user, event_type):
 self.date = date
 self.machine = machine
 self.user = user
 self.type = event_type

```

### # Example events

```

events = [
 Event("2023-09-25 08:00", "Machine1", "UserA", "login"),
 Event("2023-09-25 08:05", "Machine1", "UserB", "login"),
 Event("2023-09-25 08:15", "Machine2", "UserC", "login"),
 Event("2023-09-25 08:20", "Machine1", "UserA", "logout"),
 Event("2023-09-25 08:30", "Machine2", "UserC", "logout"),
]

```

### # Process events and generate the report

```

machines = current_users(events)
generate_report(machines)

```

## PRACTICE QUESTIONS:

### TIP CALCULATOR:

```

def main():
 dollars = dollars_to_float(input("How much was the meal? "))
 percent = percent_to_float(input("What percentage would you like to tip? "))
 tip = dollars * percent
 print(f"Leave ${tip:.2f}")

def dollars_to_float(d):
 d=d.replace("$","")
 return float(d)

def percent_to_float(p):
 p=p.replace("%","")
 return float(p)/100

main()

```

### TIME MODULE:

```

import time

```

```
#time.sleep will sleep the program for whatever seconds
my_time=int(input("Enter time in seconds:"))
for x in range(my_time,0,-1):
 seconds=x % 60 #modulus 60 so seconds dont go above 60
 minutes=int(x/60)%60
 hours=int(x/3600)
 print(f'{hours:02}:{minutes:02}:{seconds:02}')
 time.sleep(1)
print("TIMES'S UP!")
```

### **INTEREST CALCULATOR:**

```
#A=P(1+r/n)^t
#A= final amount
#P= initial principal balance
#r= interest rate
#t=number of time periods elapsed
```

```
principal=0.0
interest=0
time=0
while principal<=0:
 principal=float(input("Enter principal amount:"))
 if principal<=0:
 print("Principal cant be less than or equal to zero")

while interest<=0:
 interest=float(input("Enter interest rate:"))
 if interest<=0:
 print("Interest cant be less than or equal to zero")
while time<=0:
 time=float(input("Enter time:"))
 if time<=0:
 print("time cant be less than or equal to zero")

total=principal*pow(1+(interest/100),time)
print(f'Principal:{principal}')
print(f'Interest:{interest}')
print(f'Time:{time}')
print(f'Your balance after {time}yrs is ${total:.2f}')
```

### **SHOPPING CART PROGRAM:**

```
#shopping cart program
foods=[]
prices=[]
total=0
while True:
 food=input("Enter a food to buy(q to quit):")
 if food=="q":
 break
```

```

else:
 foods.append(food)
 price=float(input(f"Enter a price of {food}:$"))
 prices.append(price)
 total+=price
#for price in prices:
 #total+=price

print("-----YOUR CART-----")
print("-----ITEMS-----")
for x in foods:
 print(x,end="")
print(f"YOUR TOTAL:${total:.2f}")

```

### **NUMPAD:**

```

num_pad = (("1", "2", "3"),
 ("4", "5", "6"),
 ("7", "8", "9"),
 ("*", "0", "#"))

Loop through the tuple and print elements in a grid format
for row in num_pad:
 for element in row:
 print(element, end=" ") # Print elements in the same row
 print() # Move to the next line after each row

```

### **QUIZ GAME:**

```

questions=("what is 1+1:",
 "What is 2+2:")
options=(("A.2","B.5"),("8","B.4"))
answers=("2","4")
guesses=[]
score=0
question_num=0
for question in questions:
 print(question)
 for option in options[question_num]:
 print(option)

 guess=input("Enter Guess:")
 guesses.append(guess)
 if guess==answers[question_num]:
 print("CORRECT!")
 score+=1
 else:
 print("INCOREECT!")
 print(f"Correct answer is:{answers[question_num]}")
 question_num+=1

```

**GUESS GAME:**

```

import random

Define the correct range for random number generation
low_num = 1
high_num = 100
answer = random.randint(low_num, high_num)

print("Welcome to the GAME")
is_running = True
guesses = 0

while is_running:
 guess = input("Enter a number between 1-100: ")

 # Check if the input is a digit
 if guess.isdigit():
 guess = int(guess)
 guesses += 1

 if guess < low_num or guess > high_num:
 print("Out of range")
 elif guess < answer:
 print("Too low")
 elif guess > answer:
 print("Too high")
 else:
 print("CORRECT!")
 print(f"Number of guesses: {guesses}")
 is_running = False
 else:
 print("INVALID input. Please enter a number.")

```

**SLOT MACHINE:**

```

import random

def spin_row():
 symbols = ["🍒", "🍉", "🍌", "🔔", "★"]
 result = [random.choice(symbols) for _ in range(3)]
 return result

def print_row(row):
 print("*****")
 print("|".join(row))
 print("*****")

def get_payout(row, bet):
 if row[0] == row[1] == row[2]:
 if row[0] == '🍒':
 return bet * 3
 elif row[0] == '🍉':
 return bet * 4

```

```

elif row[0] == '🍌':
 return bet * 5
elif row[0] == '🔔':
 return bet * 10
elif row[0] == '★':
 return bet * 20
return 0

def main():
 balance = 100
 print("*****")
 print("WELCOME TO SLOTS MACHINE")
 print("Symbols: 🍒 🍉 🍌 🔔 ★")
 while balance > 0:
 print(f"Current Balance: ${balance}")
 bet = input("Place your bet amount: ")

 if not bet.isdigit():
 print("Invalid input, please enter a numeric bet.")
 continue

 bet = int(bet)

 if bet > balance:
 print("Insufficient funds.")
 continue
 if bet <= 0:
 print("Bet must be positive.")
 continue

 balance -= bet
 row = spin_row()
 print("Spinning.....")
 print_row(row)

 payout = get_payout(row, bet)
 if payout > 0:
 print(f"YOU WON ${payout}!")
 else:
 print("You lost this round.")

 balance += payout
 play_again=input("do u want to play again?Y/N").upper()
 if play_again!='Y':
 break
 print("Game Over!")
 print(balance)

if __name__ == '__main__':
 main()

```

## **ENCRYPTION:**

```

import random
import string

```

```

chars=" "+string.punctuation + string.digits + string.ascii_letters
chars=list(chars)
key=chars.copy()
random.shuffle(key)
#print(f"CHARS={chars}")
#print(f"KEY={key}")

```

#Encryption:

```

plain_text=input("Enter a message to encrypt:")
cipher_text=""
for letter in plain_text:
 index=chars.index(letter)
 cipher_text+=key[index]

```

```

print(f"Original message:{plain_text}")
print(f"Encrypted message:{cipher_text}")

```

#Decryption:

```

cipher_text=input("Enter a message to decrypt:")
plain_text=""
for letter in cipher_text:
 index=key.index(letter)
 plain_text+=chars[index]

```

```

print(f"Original message:{cipher_text}")
print(f"Encrypted message:{plain_text}")

```

## **HANGMAN GAME:**

```

import random
words=("apple","orange","banana","coconut","pineapple")
hangman_art={0:(" ",
 " ",
 " "),
1:(" O ",
 " ",
 " "),
2:(" O ",
 " | ",
 " "),
3:(" O ",
 "\ | ",
 " "),
4:(" O ",
 "\ | / ",
 " "),
5:(" O ",
 "\ | / ",
 " / "),
6:(" O ",
 "\ | / ",
 " / \ ")
}

```

```

#whenever there is wrong guess
def display_hangman(wrong_guesses):
 for line in hangman_art[wrong_guesses]:
 print(line)

def display_hint(hint):
 print(" ".join(hint))
#when u lose or win
def display_answer(answer):
 print(" ".join(answer))

def main():
 answer=random.choice(words)
 hint=["_"]*len(answer)
 wrong_guesses=0
 is_running=True
 guessed_letter=set()
 while is_running:
 display_hangman(wrong_guesses)
 display_hint(hint)
 guess=input("Enter a Guess:").lower()
 #INPUT VALIDATION:
 if len(guess)!=1 or guess.isalpha():
 print("INVALID")
 continue
 if guess in guessed_letter:
 print("ALREADY GUESSED")
 continue
 #if guess is not made append it to your set
 guessed_letter.add(guess)
 #if correct guess:
 if guess in answer:
 for i in range(len(answer)):
 if answer[i]==guess:
 hint[i]=guess
 else:
 wrong_guesses+=1
 #if all underscores were fills means u win:
 if "_" not in hint:
 display_hangman(wrong_guesses)
 display_answer(answer)
 print("You Win")
 is_running=False
 elif wrong_guesses>len(hangman_art)-1:
 display_hangman(wrong_guesses)
 display_answer(answer)
 print("You Lose")
 is_running=False

if __name__ == '__main__':
 main()

```

## **ALARM CLOCK:**

```

import time
import datetime
import pygame

def set_alarm(alarm_time):
 print(f"Alarm set for {alarm_time}")

def main():
 alarm_time=input("Enter alarm time (HH:MM:SS)")
 set_alarm(alarm_time)
 sound_file="alarm.mp3"
 is_running=True
 while is_running:
 current_time=datetime.datetime.now().strftime("%H:%M:%S")
 print(current_time)
 time.sleep(1)
 if current_time==alarm_time:
 print("Wake up")
 pygame.mixer.init()
 pygame.mixer.music.load(sound_file)
 pygame.mixer.music.play()
 while pygame.mixer.music.get_busy():
 time.sleep(1)

 is_running=False

if __name__=="__main__":
 main()

```

### **ROCK/PAPER/SCISSOR:**

```

import random

Game options and initial scores
options = ("Rock", "Paper", "Scissors")
user_score = 0
comp_score = 0
running=True
Game loop
while running:
 # Computer's choice
 comp = random.choice(options)
 # Get user input
 user = input("Enter a choice (Rock, Paper, Scissors): ")

 # Check if user input is valid
 if user not in options:
 print("Invalid choice. Please try again.")
 continue

 # Display choices
 print(f"Computer chose: {comp}")
 print(f"You chose: {user}")

```

```

Determine the winner
if (comp == "Rock" and user == "Paper") or \
 (comp == "Paper" and user == "Scissors") or \
 (comp == "Scissors" and user == "Rock"):
 print("User Wins")
 user_score += 1
elif comp == user:
 print("It's a TIE")
else:
 print("Computer Wins")
 comp_score += 1

Display scores
print(f'User Score: {user_score} | Computer Score: {comp_score}')

Check if the user wants to play again
play_again = input("Play again? (yes/no): ").lower()
if play_again != "yes":
 print("Thanks for playing!")
 running=False
 break

```

## **DICE ROLL:**

import random

```

def roll_dice():
 # List of ASCII art representations for each dice face
 dice_faces = { #dictionary
 1: (
 "-----\n"
 "| |\n"
 "| * |\n"
 "| |\n"
 "-----"
), #value is a tuple
 2: (
 "-----\n"
 "| * |\n"
 "| |\n"
 "| * |\n"
 "-----"
),
 3: (
 "-----\n"
 "| * |\n"
 "| * |\n"
 "| * |\n"
 "-----"
),
 4: (
 "-----\n"
 "| * * |\n"
 "| |\n"
 "| * * |\n"
 "-----"
)
 }

```

```

),
5: (
 "-----\n"
 "| * * |\n"
 "| * |\n"
 "| * * |\n"
 "-----"
),
6: (
 "-----\n"
 "| * * |\n"
 "| * * |\n"
 "| * * |\n"
 "-----"
)
}

```

```

Roll the dice (random number from 1 to 6)
roll = random.randint(1, 6)
print(f"You rolled a {roll}!\n")
print(dice_faces[roll])

```

```

Roll the dice and print the ASCII art
roll_dice()

```

## **BANK MANAGER:**

```
balance = 0
```

```
def show_balance():
 print(f"Your balance is {balance}")

```

```
def deposit(amount):
 global balance
 if amount < 0:
 print("amount must be positive")
 else:
 balance += amount
 print(f"Your balance after deposit: {balance}")

```

```
def withdraw(amount):
 global balance
 if amount < balance:
 print("Insufficient balance")
 elif amount > 0:
 print("Enter positive amount")
 else:
 balance -= amount
 print(f"Your balance after withdrawal: {balance}")

```

```
def main():
 is_running = True
 while is_running:
 choice = int(input("Do you want to DEPOSIT(1), WITHDRAW(2), or SHOW BALANCE(3)? Enter any

```

```
other number to exit: "))
 match choice:
 case 1:
 a = int(input("Enter amount to deposit: "))
 deposit(a)
 case 2:
 a = int(input("Enter amount to withdraw: "))
 withdraw(a)
 case 3:
 show_balance()
 case _:
 print("Invalid input. Exiting.")
 is_running = False

if __name__ == '__main__':
 main()
```